เอกสารพกเข้าห้องสอน · แบบเปิดตำรา (Open Book)

📘 เอกสารพกเข้าห้องสอบ Final — Operating Systems (ระบบปฏิบัติการ)

05506007 · เรียบเรียงจาก Presentation ของอาจารย์ (ภาพต้นฉบับ) + Handout + วิดีโอ + Quizizz ในห้องเรียน

เน้นละเอียดบทที่ 7–13 · พิมพ์เป็น A4 ได้ (Ctrl+P) · ภาพสไลด์ทุกภาพอ้างอิงเลขสไลด์จริงจาก presentation ของอาจารย์

🖨️ วิธีใช้ + พิมพ์

• เปิดหน้านี้ทิ้งไว้ กด `Ctrl+P` → เลือก Save as PDF → เปิด "Background graphics" เพื่อให้สีสไลด์/ตารางออกครบ

• กล่อง **แดง = จุดหลอกข้อสอบ** · **เขียว = เทคนิค/สูตรทำเร็ว** · **ม่วง = อุปมาช่วยจำ** · ภาพสไลด์ = ข้อมูลต้นฉบับของอาจารย์ (ตัวเลขในโจทย์ตัวอย่างตรงกับสไลด์)

สารบัญ

A. Process (กระบวนการ) & fork (แตกสำเนากระบวนการ)/exec (เปลี่ยนโปรแกรมที่กำลังรัน)/wait (รอ + หยิบสิทธิ์) 🔥

B. Thread (เธรด (สายงานย่อย)) & Multi-Threading

C. CPU Scheduling (การจัดคิว CPU) — Gantt/TAT/WT 🔥

D. Race Condition (สภาพแข่งขันของข้อมูล) & Critical Section (ช่วงวิกฤต (โค้ดแตะของแชร์))

E. Semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง) · Monitor (กล่องควบคุมระดับสูง) · Java (ภาษาจาวา) sync 🔥

F. Deadlock (การติดตาย (รอกันตลอดกาล)) + Banker's Algorithm (อัลกอริทึมนายธนาคาร) 🔥

G. Real Memory (หน่วยความจำ) Management

H. Paging (การแบ่งหน่วยความจำเป็นหน้า) & Address (ที่อยู่หน่วยความจำ) Translation 🔥🔥

I. Virtual Memory & Page Replacement (การเลือกเพจออกเมื่อ RAM เต็ม) 🔥🔥

J. File System (ระบบไฟล์)

K. ความรู้จาก Quizizz ทั้ง 4 ชุด

L. แผ่นสูตรเร็วท้ายเล่ม (1 หน้า)

A. Process & การสร้าง Process ด้วย fork / exec / wait (OSNo03)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: Process = โปรแกรมที่ "มีชีวิต" กำลังทำงานอยู่ใน RAM มีสถานะและทรัพยากรของตัวเอง · fork = คำสั่งแตกสำเนา process (กระบวนการ), exec = เปลี่ยนโปรแกรมที่ process กำลังรัน, wait = ให้พ่อรอลูกจบ

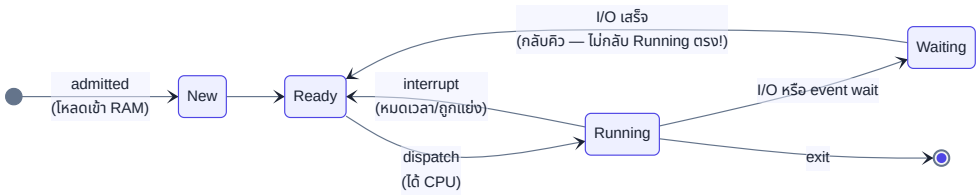
ความสำคัญกับข้อสอบ: ข้อสอบชอบให้ "ไล่โค้ด fork" — นับว่ามี process เกิดกี่ตัว เดาว่า output พิมพ์อะไร รุ่นพี่เตือนว่ามีจุดหลอกเยอะ และย้ำว่าโค้ดทุกข้อ compile ผ่านหมด ไม่มีคำตอบว่า Error

ใช้จริงในคอมพิวเตอร์: ทุกครั้งที่พิมพ์คำสั่งใน terminal หรือดับเบิลคลิกเปิดแอป คือ shell (โปรแกรมรับคำสั่งผู้ใช้) ทำ fork → exec → wait ให้เราทั้งหมด — เช่น พิมพ์ ls, เปิด Chrome, compile ไฟล์

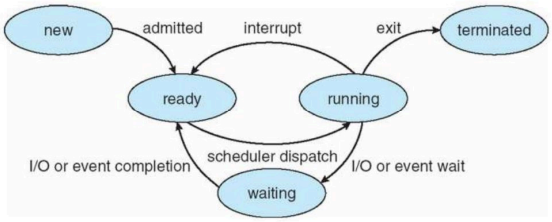
A1. Program vs Process · สถานะของ Process

	Program	Process
คือ	ไฟล์คำสั่งนิ่ง ๆ บนดิสก์ — **passive entity**	**program in execution** — active entity ต้องใช้ทรัพยากร
มีอะไร	แค่โค้ด	Program counter (PC), stack, data (ข้อมูล) section, heap, registers
ชื่อเรียกอื่น	—	Task (time-sharing) · Job (batch system)

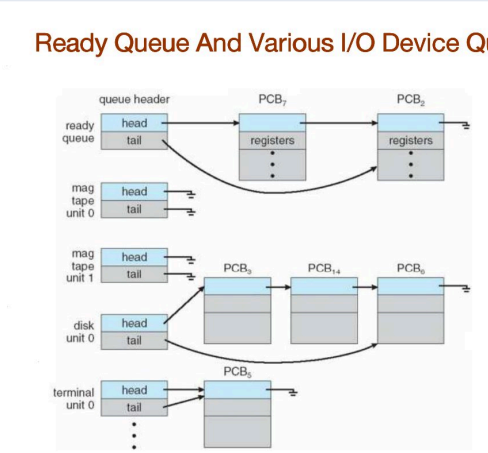
สถานะของ process (state diagram):



> **🎯 จุดหลอก**
>
> Waiting เสร็จแล้วไป **Ready ก่อนเสมอ** ไม่ใช่กลับ Running ทันที (ต้องรอ dispatcher เรียก) · Batch system เรียก process ว่า Job, time-sharing เรียก Task



สไลด์ OSNo03 p.4 — Diagram of Process State (ต้นฉบับอาจารย์)



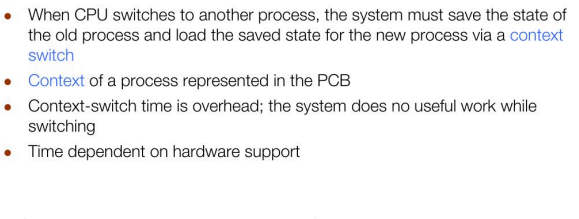
สไลด์ OSNo03 p.7 — Ready Queue (คิวงานที่พร้อมรัน) และ I/O Device Queues: process วิ่งวน Ready → CPU → I/O คิว

A2. PCB — Process Control Block (บล็อก (หน่วยจัดเก็บ))

PCB = "รหัสประชาชน" ของ process เก็บทุกอย่างที่ OS ต้องรู้: **process state, program counter, CPU registers, scheduling (การจัดคิว) info (priority (ลำดับสิทธิ์)/queue), memory (หน่วยความจำ) info (base/limit), accounting info, I/O status**



สไลด์ OSNo03 p.8 — Process Control Block



สไลด์ OSNo03 p.9 — Context Switch (การสลับงานบน CPU): เซฟ state ของ P0 ลง PCB0 → โหลด PCB1 → รัน P1 (ช่วงนี้ = pure overhead)

> **Dispatcher (ผู้ส่งงานเข้า CPU) ทำ 3 อย่าง**
>
> (1) switching context (2) switch to user mode (3) jump ไปตำแหน่งเดิมใน user program · **Dispatch latency** = เวลาที่ dispatcher ใช้หยุดตัวหนึ่ง เริ่มอีกตัวหนึ่ง

A3. fork() — สร้าง process แบบโคลนตัวเอง

`pid_t fork(void)` สร้าง child โดย**ก๊อปปี้ parent** (address (ที่อยู่หน่วยความจำ) space, ตัวแปร, fd ทุกอย่าง) แล้วทั้งคู่รันต่อจากบรรทัดเดียวกัน (หลัง fork)

ใครเรียก	ค่าที่ fork() คืน	เงื่อนไข if
ใน Parent	**PID ของลูก** (> 0)	`if (pid > 0)`
ใน Child	**0**	`else if (pid == 0)`
สร้างไม่สำเร็จ	**ค่าลบ**	`if (pid < 0)` → error

🪄 เข้าใจง่าย

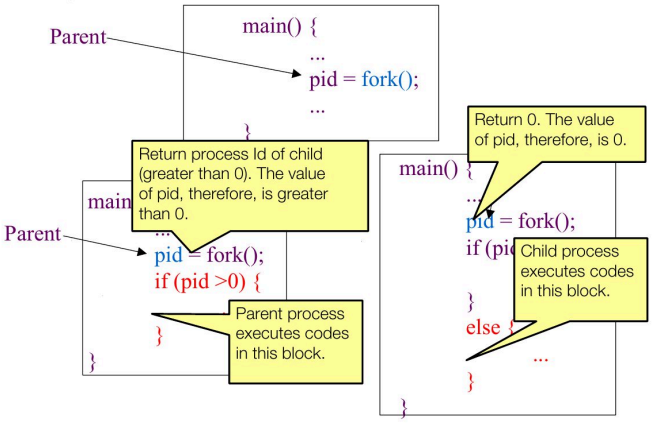
fork() เรียก**ครั้งเดียวแต่คืนค่า 2 ครั้ง** — ครั้งนึงใน "โลกพ่อ" ครั้งนึงใน "โลกลูก" ทั้งคู่ถือ copy ของตัวแปร pid ของตัวเอง แก้ฝั่งไหนไม่กระทบอีกฝั่ง

```c
/* fork_example.c — ตัวอย่างจากสไลด์/วิดีโอ: ผลลัพธ์พิมพ์ 2 บรรทัด */
#include <stdio.h>
#include <sys/types.h>   // pid_t
#include <unistd.h>      // fork()

int main(void) {
    pid_t pid;                  // เก็บค่า return ของ fork
    int i = 0;                  // ตัวแปรธรรมดา — จะถูก "ก๊อป" ไปให้ลูกด้วย

    pid = fork();               // ← จุดแตกหน่อ: หลังบรรทัดนี้มี 2 process รันโค้ดชุดเดียวกัน
    if (pid > 0) {              // โลกพ่อ: ได้ PID ของลูก (>0) เสมอ
        i = 1;                  // แก้ i ใน address space ของพ่อ — ลูกไม่เห็น
        printf("I am parent i = %d\n", i);
    }
    else {                      // โลกลูก: fork คืน 0
        i = 2;
        printf("I am child i = %d\n", i);
    }
    return 0;
}
/* Output: "I am parent i = 1" และ "I am child i = 2" (2 บรรทัดเสมอ)
   แต่ลำดับก่อน-หลัง non-deterministic — ขึ้นกับ scheduler ห้ามสรุปใครพิมพ์ก่อน */
```

Diagram for fork() example



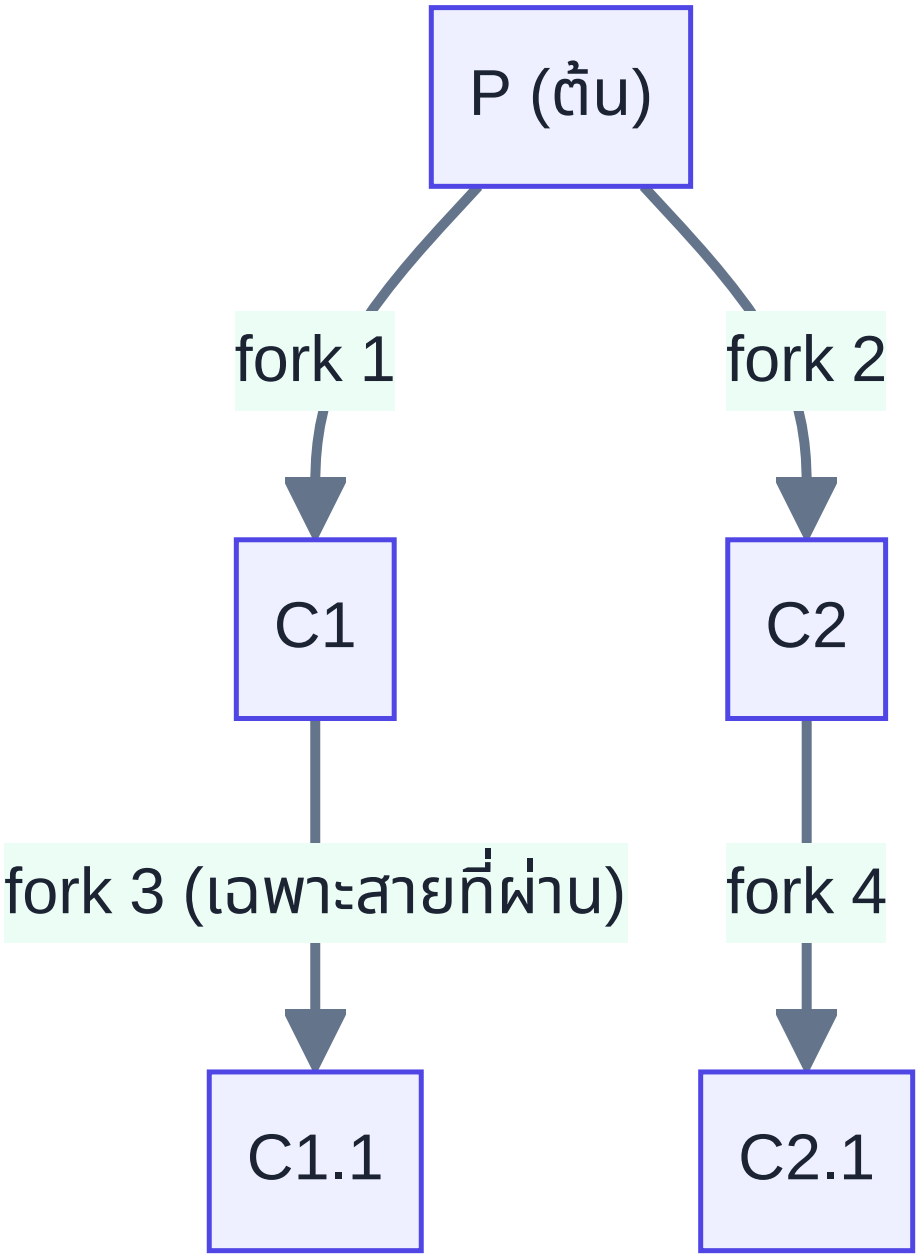
สไลด์ OSNo03 p.19 — Diagram for fork() example: พ่อ/ลูกรันต่อจากจุดเดียวกัน แยกคนละสาย

Process IDs (pids)

- pid = fork();
- In the child: pid == 0;
  In the parent: pid == the process ID of the child.
- If the value of pid is negative, it indicates that the creation of a child process was unsuccessful.
- The application can utilize this difference in the value of pid to execute distinct actions in the parent and child processes.

สไลด์ OSNo03 p.20 — Process IDs: ลูกได้ 0 / พ่อได้ PID ลูก / ติดลบ = fail

A3.1 นับจำนวน process — วาด Process Tree (แผนภาพต้นไม้ความสัมพันธ์พ่อ-ลูก)



`fork()` ติดกัน `n` ครั้ง ไม่มีเงื่อนไข → process ทั้งหมด = 2^n (ลูกใหม่ = $2^n - 1$)

fork 3 ครั้ง = 8 process · ถ้ามี if/exit ให้วาด tree แล้วนับ "สายที่ผ่าน fork จุดนั้น" ทีละจุด

A3.2 fork + loop จากวิดีโอ (จุดหลอก: ลูกต้อง exit)

```
for (i = 0; i < 5; i++) {          // พ่อวน 5 รอบ
    pid = fork();
    if (pid == 0) {                 // ลูกเท่านั้นเข้า if
        if (i % 2 != 0) printf("I am happy\n");  // ลูกที่เกิดตอน i คี่
        else            printf("I am sad\n");    // ลูกที่เกิดตอน i คู่
        exit(0);                    // ← ถ้าลืม! ลูกจะหลุด if ไปวน for ต่อ
    }                               //   แล้ว fork ลูก/หลานของตัวเองเพิ่ม
}
while (wait(NULL) != -1);           // พ่อรอลูกทุกตัว (wait คืน -1 เมื่อไม่มีลูกเหลือ)
printf("I am parent goodbye\n");
```

🎯 จุดหลอก (อาจารย์สาธิตในวิดีโอ)

มี exit() → sad ×3 (i=0,2,4), happy ×2 (i=1,3), goodbye **1 ครั้งสุดท้าย** · ถ้าลูกเลขคี่**ลืม exit()** → ลูก C1, C3 กลายเป็น "พ่อ" สร้างหลาน → goodbye พิมพ์ **4 ครั้ง** (พ่อหลัก + C1 + C3 + หลาน $C1_{-3}$)

A4. exec() — เปลี่ยนร่าง ไม่สร้างคนใหม่

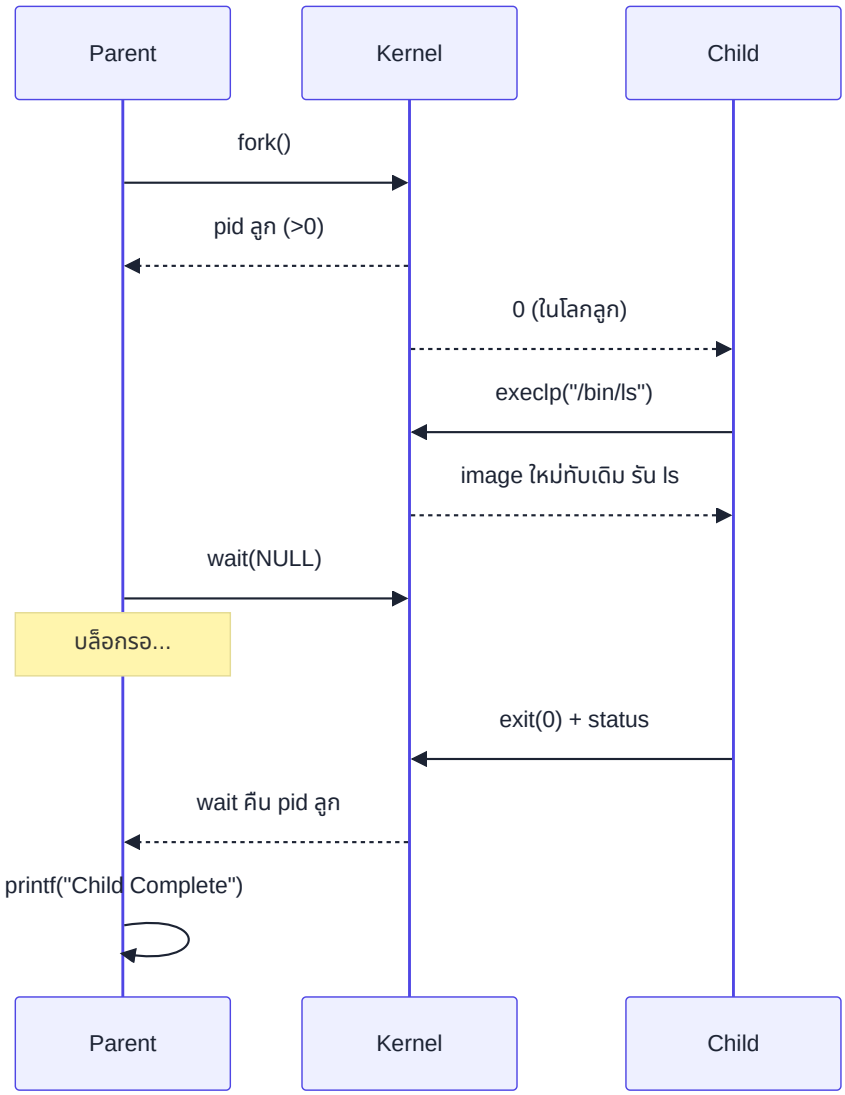
- ตระกูล exec แทนที่ **process image ทั้งหมด** (โค้ด+data) ด้วยโปรแกรมใหม่ — PID คงเดิม
- `execlp(file, arg0, arg1, ..., NULL)` — **l** = list (ส่งทีละตัว) · **p** = หาโปรแกรมตาม PATH · **arg0** = ธรรมเนียมใส่ชื่อโปรแกรม (exec ข้าม แต่ต้องใส่) · **NULL ปิดท้ายบังคับ**
- execv: ส่งเป็น array `char *argv[]`

🎯 จุดหลอก exec

exec **สำเร็จ** → โค้ดหลังบรรทัด exec **ไม่มีวันรัน** (image โดนทับหมด) · exec **ล้มเหลว** (ชื่อผิด เช่น "ly") → รันบรรทัดถัดไปต่อ · `exec("cd")` fail เสมอ เพราะ **cd เป็น shell built-in** ไม่ใช่ไฟล์ (echo ได้เพราะมี /usr/bin/echo)

A5. fork + exec + wait — รูปแบบมาตรฐานของ shell

```c
int main() {
    pid_t pid = fork();
    if (pid < 0) {                          // สร้างลูกไม่สำเร็จ
        fprintf(stderr, "Fork Failed");
        exit(-1);
    }
    else if (pid == 0) {                    // ── ลูก: เปลี่ยนร่างเป็น ls ──
        execlp("/bin/ls", "ls", NULL);      // สำเร็จ = โค้ดลูกถูกแทนที่ทั้งหมด
    }
    else {                                  // ── พ่อ: รอลูก ──
        wait(NULL);                          // บล็อกรอลูกตัวใดตัวหนึ่งจบ (ได้ pid ลูกกลับ)
        printf("Child Complete");           // พิมพ์หลังลูกจบเสมอ
        exit(0);
    }
}
/* Output: ผล ls ก่อน ตามด้วย "Child Complete" — ลำดับแน่นอนเพราะ wait() */
```



A6. exit / wait / waitpid (รอลูกที่ระบุตัวเจาะจง) + สถานะพิเศษ

Call	ทำอะไร	จุดสำคัญ
`exit(status)`	ขอ OS ทำลายตัวเอง ส่ง status ให้พ่อ	ส่งได้แค่ **8 บิต (0–255)** — exit(300) พ่ออ่านได้ 300 mod 256 = 44
`wait(&status)`	รอ*ลูกใดก็ได้*จบ	คืน pid ลูกที่จบ · ไม่มีลูก → คืน **-1** · ไม่สน status ใช้ wait(NULL)
`waitpid(pid, &status, 0)`	รอ*ลูกตัวเจาะจง*	พ่อรายงานผลเรียงลำดับได้ (ต่างจาก wait ที่ได้ใครมาก่อน)
`WIFEXITED(status)`	เช็คลูกจบแบบปกติ?	พิมพ์ status ตรง ๆ ได้เลขแปลก (32768 ฯลฯ) เพราะ OS แปะข้อมูลอื่นปน — **ต้อง decode ด้วยแมโคร**
`WEXITSTATUS(status)`	ดึงค่า exit จริง	

🧟 Zombie (กระบวนการที่ตายแล้วแต่รายการยังค้างในระบบ) process

ลูก**ตายแล้ว**แต่พ่อไม่ wait() → ทรัพยากรถูกคืนแต่รายการใน process table (ตาราง) ค้าง สถานะ **Z / defunct** — ทางแก้: พ่อต้อง wait

👶 Orphan (กระบวนการที่พ่อตายก่อน) process

พ่อ**ตายก่อนลูก** → OS ให้ init/systemd "รับเลี้ยง" (ps -l เห็น PPID เปลี่ยน) · **Cascading termination**: พ่อโดน terminate แล้ว OS บางระบบลบลูกหลานตามทั้งตระกูล

A6.1 ประสานพ่อ-ลูก: wait vs waitpid (จากวิดีโอ parent.c/child.c)

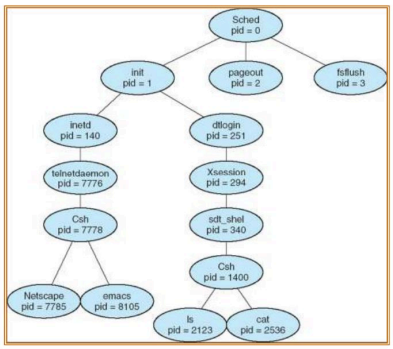
```c
/* child.c: รับ pid ตัวเอง สุ่ม sleep 0-4 วิ แล้ว exit(pid % 256)
   parent.c: fork 3 ลูก (pid[i]) แล้วเปรียบเทียบสองแบบ */
// แบบ 1: wait(&status) → ลูกตัวไหนจบก่อน พ่อรายงานตัวนั้นก่อน (ลำดับไม่แน่)
while ((pid = wait(&status)) > 0) printf("child %d done\n", pid);
// แบบ 2: waitpid(pid[i], &status, 0) → พ่อรอ "ตามลำดับที่กำหนด" เสมอ
for (i = 0; i < 3; i++) {
    waitpid(pid[i], &status, 0);   // นิ่งรอลูกตัวที่ 1 ก่อน แม้ตัว 3 จบก่อน
    printf("child %d done\n", pid[i]);
}
```

Fork and exec()

```c
int main()
{
    pid_t  pid;
    /* fork another process */
    pid = fork();
    if (pid < 0) { /* error occurred */
            fprintf(stderr, "Fork Failed");
            exit(-1);
    }
    else if (pid == 0) { /* child process */
            execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
            /* parent will wait for the child to complete */
            wait (NULL);
            printf ("Child Complete");
            exit(0);
    }
}
```

สไลด์ OSNo03 p.23 — Fork and exec(): โครงโค้ดมาตรฐานจากสไลด์อาจารย์

A tree of processes on a typical Solaris



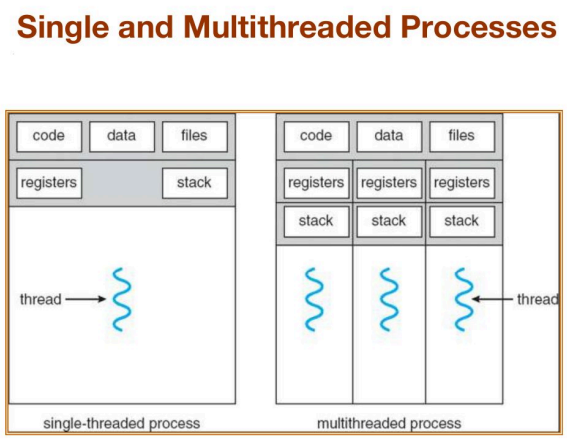
สไลด์ OSNo03 p.15 — Process tree (login → sh → คำสั่งต่าง ๆ)

B. Thread & การเขียนโปรแกรม Multi-Threaded (OSNo04)

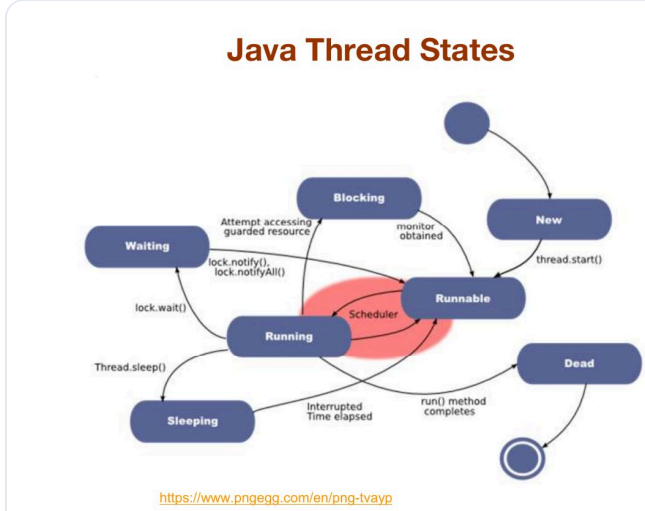
> 📖 **รู้จักส่วนนี้ก่อนอ่าน**
>
> **ความหมาย:** Thread (เธรด/สายงานย่อย) = หน่วยทำงานที่เล็กกว่า process หลาย thread (เธรด (สายงานย่อย)) รันขนานกันภายใน process เดียวกันได้ โดยแชร์หน่วยความจำร่วมกัน
>
> **ความสำคัญกับข้อสอบ:** ถามโมเดลการ map user thread (thread ฝั่งผู้ใช้) → kernel thread (Many-to-One / One-to-One / Many-to-Many) พร้อมตัวอย่าง OS, อ่านโค้ด Pthread/Java ว่าอะไรแชร์กัน และเชื่อมไปบท Synchronization (การประสานงาน)
>
> **ใช้จริงในคอมพิวเตอร์:** เบราว์เซอร์ 1 แท็บใช้หลาย thread (โหลดหน้าเว็บ + เล่นวิดีโอ + รับการคลิก พร้อมกัน) · เกมออนไลน์ (เครือข่าย/กราฟิก/ฟิสิกส์แยกกัน) · Thread Pool (บ่อรวมเธรดเตรียมใช้งาน) ใช้ในเว็บเซิร์ฟเวอร์รับคำขอนับพันพร้อมกัน

B1. Thread คืออะไร

Thread = ลำดับคำสั่งที่รันได้อิสระภายใน process เดียวกัน (lightweight process) · สิ่งที่แต่ละ thread มี**ส่วนตัว**: **program counter, registers, stack** · สิ่งที่**แชร์กัน**: code section, data section, heap, ไฟล์ที่เปิด — thread คือ "หน่วยที่ OS จัดคิวเข้า CPU" ในระบบสมัยใหม่ และ**สลับ thread ถูกกว่าสลับ process**



สไลด์ OSNo04 p.4 — Single vs Multithreaded: ขวาคือแชร์ code/data/files ร่วมกัน แยกแค่ registers+stack ของแต่ละ thread



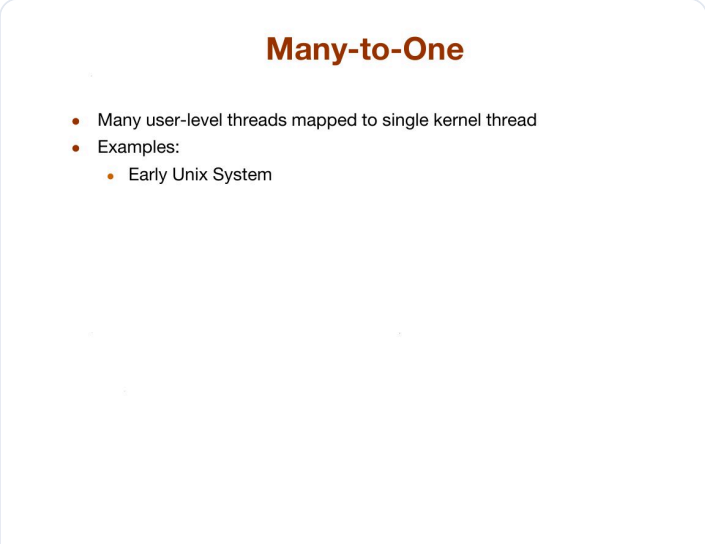
สไลด์ OSNo04 p.25 — Java Thread States (New/Runnable/Running/Blocked/Terminated)

OS แบบไหน	ตัวอย่าง
Single-process single-threaded	**MS-DOS**
Single-process multi-threaded	**JVM**
Multiprocessing single-threaded	Traditional UNIX (ระบบยูนิกซ์)
Multiprocessing multi-threaded	**Modern OS: Windows (ระบบวินโดวส์), Linux (ระบบลินุกซ์), macOS**

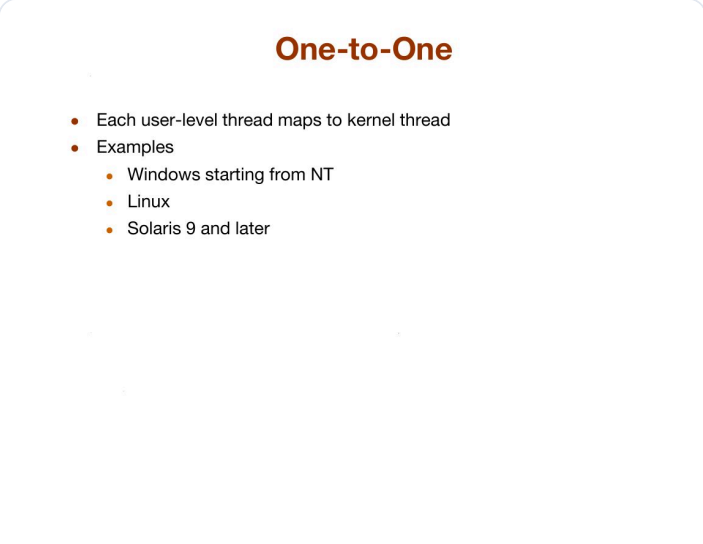
B2. User Thread vs Kernel (แกนหลักของ OS) Thread + Multithreading Models

- **User-Level Thread (ULT)**: จัดการโดย thread library ฝั่งผู้ใช้ kernel ไม่รู้จัก — เร็วมาก แต่ถ้าตัวใด block (บล็อก (หน่วยจัดเก็บ)) ทั้ง process หยุด
- **Kernel-Level Thread (KLT)**: kernel จัดการตรง — block ตัวเดียวตัวอื่นยังรัน ขนานจริงได้

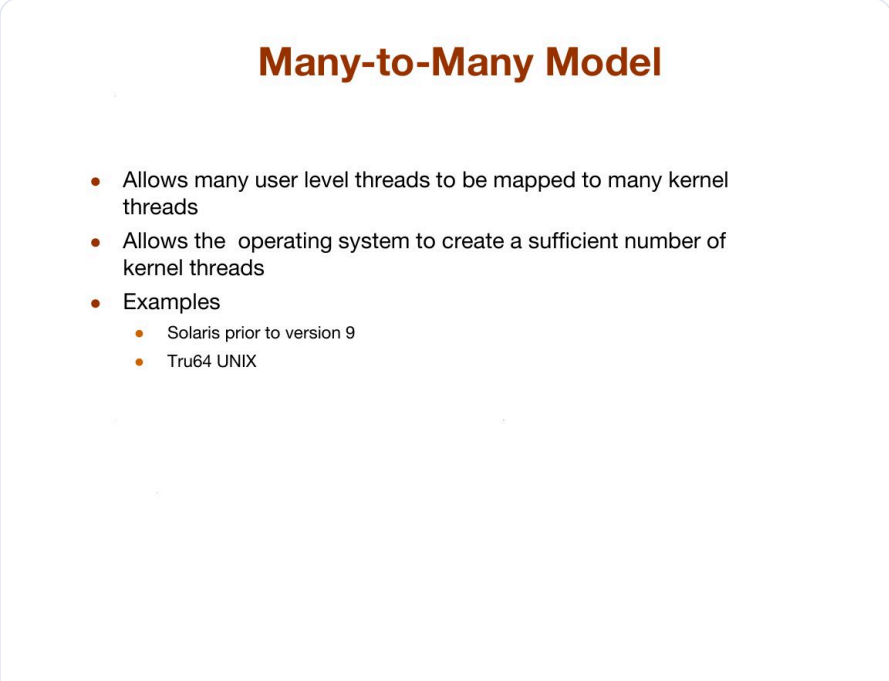
Model	Map	จุดเด่น/จุดพัง	ตัวอย่าง
Many-to-One	ULT หลาย → KLT 1	เร็ว (user space) แต่ block ตัวเดียว = หยุดทั้ง process, ขนานบนหลาย CPU ไม่ได้	Early Unix (ระบบยูนิกซ์)
One-to-One	ULT 1 : 1 KLT	ขนานจริง ตัวใด block ตัวอื่นไม่พัง · สร้าง kernel thread เยอะ = overhead	**Windows NT+, Linux, Solaris 9+**
Many-to-Many	ULT หลาย → KLT หลาย	OS สร้าง KLT ให้พอดีจำนวน	Solaris < 9, Tru64 UNIX



p.13 Many-to-One



p.15 One-to-One



p.17 Many-to-Many

B3. Pthreads (C) — โค้ดตัวอย่างพร้อมคอมเมนต์

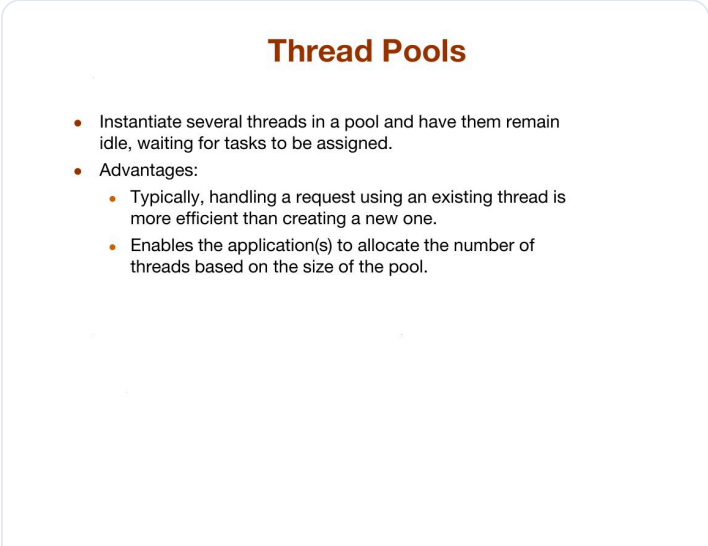
```c
int sum;  // ตัวแปร global — "แชร์" ระหว่าง thread (main กับ runner เห็นตัวเดียวกัน!)

/* ฟังก์ชันที่ thread จะรัน — signature ต้องเป็น void *f(void *) */
void *runner(void *param) {
    int upper = atoi(param);      // รับค่าจาก argv[1] ที่ส่งมาตอน create
    sum = 0;
    for (int i = 1; i <= upper; i++) sum += i;   // หาผลรวม 1..upper
    pthread_exit(NULL);           // จบ thread นี้ (≈ exit ของ process)
}

int main(int argc, char *argv[]) {
    pthread_t tid;                // เก็บ id ของ thread ที่สร้าง
    pthread_attr_t attr;
    pthread_attr_init(&attr);     // ตั้งค่า attribute = default
    pthread_create(&tid, &attr, runner, argv[1]);
    //  ↑ สร้าง thread ให้รันฟังก์ชัน runner โดยส่ง argv[1] เป็น param
    pthread_join(tid, NULL);      // หลักของ main: รอ thread จบก่อน (≈ wait ของ process)
    printf("sum = %d\n", sum);    // เห็นค่า sum ที่ runner แก้ — เพราะแชร์กันจริง
}
/* จับคู่: pthread_create ≈ fork · pthread_join ≈ wait · pthread_exit ≈ exit */
```



สไลด์ OSNo04 p.19 — Pthreads (โค้ดต้นฉบับ)



สไลด์ OSNo04 p.29 — Thread Pools: สร้าง thread เตรียมไว้ใน "บ่อ" รับงานเรื่อย ๆ

Thread Pool (Java 3 แบบ)

Single thread executor (บ่อขนาด 1) · **Fixed thread executor** (ขนาดคงที่) · **Cached thread pool** (ขนาดไม่จำกัด สร้างเพิ่มเอง) — ประโยชน์: ใช้ thread เดิมเร็วกว่าสร้างใหม่ + คุมจำนวน thread ได้

B4. Java Thread — 2 วิธีสร้าง + cancellation

วิธี	โค้ด	หมายเหตุ
extends Thread	`class W extends Thread { public void run() {...} }` → `new W().start()`	เผา slot extends
implements Runnable	`class W implements Runnable { public void run() {...} }` → `new Thread(new W()).start()`	นิยมกว่า

- **Cancellation 2 แบบ: Asynchronous (ยกเลิกทันที)** = ยิงปิดทันที (เสี่ยงข้อมูลครึ่ง ๆ กลาง ๆ) · **Deferred (ยกเลิกแบบให้ตัวมันเช็คเองเป็นระยะ)** = บอกเป้าหมายให้เช็คเองเป็นระยะ (`Thread.currentThread().isInterrupted()`) แล้ว cleanup ก่อนจบ — ปลอดภัยกว่า
- `task.join()` = main รอ task จบก่อนเสมอ (เทียบ wait)

🎯 ประโยคทอง Java scheduling

Java: priority **1–10, default (ค่าเริ่มต้น) 5, มาก = สูง** (Unix ตรงข้าม: น้อย = สูง) · การตั้ง priority **ไม่การันตีลำดับ 100%** — **ห้ามฝากความถูกต้องของ logic ไว้กับ scheduler (ตัวจัดคิว CPU)** ต้องใช้ Synchronization เท่านั้น

C. CPU Scheduling — Gantt · TAT · WT (OSNo04 สไลด์ท้าย + วิดีโอ 6.x)

> 📖 **รู้จักส่วนนี้ก่อนอ่าน**
>
> **ความหมาย:** CPU Scheduling = การตัดสินใจว่า process ไหนจะได้ใช้ CPU "ตัวต่อไป" เมื่อหลายงานแย่งกัน — มีหลายอัลกอริทึม เช่น FCFS (มาก่อนได้ก่อน), SJF (งานสั้นก่อน), RR (วนแบ่งเวลา)
>
> **ความสำคัญกับข้อสอบ:** โจทย์วาด Gantt (แผนภูมิช่วงเวลา) หา TAT/WT เป็นข้อคำนวณหลักประจำทุกปี — ฝึกให้ทำข้อละไม่เกิน 2 นาที ใช้เทคนิค "หา TAT ก่อนแล้วลบ BT เสมอ"
>
> **ใช้จริงในคอมพิวเตอร์:** Windows/Linux/macOS ใช้แนวคิด priority + time slice (โควตาเวลาแบบ RR) จริง ๆ ในตัวจัดคิวของเคอร์เนล — เปิด Task Manager เห็น process แย่ง CPU ด้วยกลไกพวกนี้ทั้งหมด

> 📖 **รู้จักส่วนนี้ก่อนอ่าน**
>
> **ความหมาย:** CPU Scheduling = การตัดสินใจว่า process ไหนจะได้ใช้ CPU "ตัวต่อไป" เมื่อมีหลายงานแย่งกัน — มีหลายอัลกอริทึม เช่น FCFS (มาก่อนได้ก่อน), SJF (งานสั้นก่อน), RR (วนแบ่งเวลา)
>
> **ความสำคัญกับข้อสอบ:** โจทย์คำนวณวาด Gantt (แผนภูมิช่วงเวลา) แล้วหา TAT/WT เป็นข้อคำนวณหลักประจำทุกปี — ฝึกให้ทำข้อละไม่เกิน 2 นาที ใช้เทคนิค "หา TAT ก่อนแล้วลบ BT"
>
> **ใช้จริงในคอมพิวเตอร์:** Windows/Linux/macOS ใช้แนวคิด priority + time slice (โควตาเวลาแบบ RR) จริง ๆ ในตัวจัดคิวของเคอร์เนล — เปิด Task Manager เห็น process แย่ง CPU ตามกลไกพวกนี้ทั้งหมด

C1. พื้นฐาน + สูตรทุกตัวที่ใช้คิดเลข

- **CPU Scheduler (ตัวจัดคิว CPU)** เลือก process จาก ready queue ให้ใช้ CPU · **Non-preemptive** = รันจนจบ/ขอออกเอง · **Preemptive (ขัดจังหวะ/แย่งได้)** = ขัดกลางคันได้ (หมดเวลา/มีตัวสำคัญกว่ามา)
- **CPU burst** = ช่วงที่ process ใช้ CPU ติดกัน · I/O-bound = burst สั้นถี่ / CPU-bound = burst ยาว

`Turnaround Time (เวลารวมในระบบ): TAT = Completion − Arrival`

เวลารวมตั้งแต่งานมาถึง → เสร็จ

`Waiting Time (เวลารอคิว): WT = TAT − Burst Time (เวลาที่ต้องใช้ CPU)`

🟢 เทคนิคทำเร็ว: หา TAT ก่อนแล้วลบ BT เสมอ — โดยเฉพาะ preemptive ที่งานโดนแทรกหลายรอบ

`Response Time (เวลาตอบสนอง) = เวลาที่งานมาถึง → เริ่มได้ response ครั้งแรก`

เกณฑ์วัดอื่น: CPU utilization ↑, throughput ↑, TAT ↓, WT ↓, response ↓

C2. FCFS — First-Come, First-Served (non-preemptive)

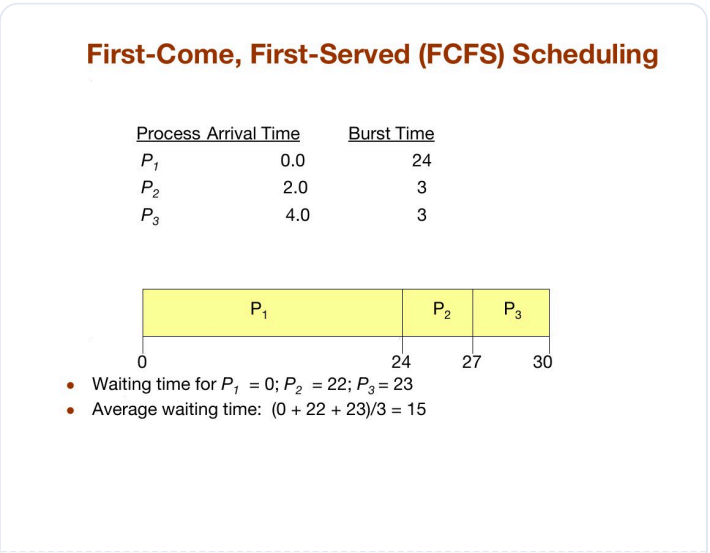
โจทย์ (ตรงจากสไลด์ p.34): P1 (Arrival 0, Burst 24) · P2 (2, 3) · P3 (4, 3)



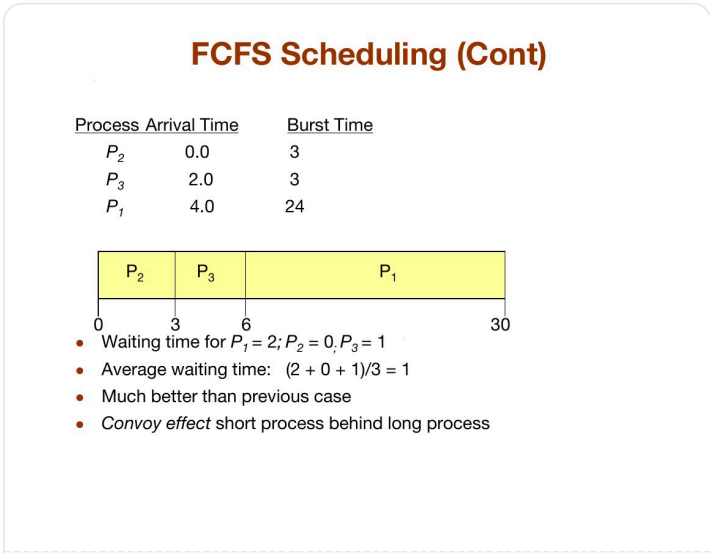
Process	TAT = C − A	WT = TAT − BT
P1	24 − 0 = 24	24 − 24 = **0**
P2	27 − 2 = 25	25 − 3 = **22**
P3	30 − 4 = 26	26 − 3 = **23**
เฉลี่ย	**25**	**(0+22+23)/3 = 15**

🎯 Convoy Effect (อาการคอยตามงานใหญ่)

งานใหญ่ตัวเดียวบล็อกหน้า งานเล็กทั้งบ้านต่อแถวรอ — สไลด์ p.35 เปรียบเทียบ: ถ้างานสั้นมาก่อน (P2 มา 0, P3 มา 2, P1 มา 4) avg WT เหลือ **1** เท่านั้น!



สไลด์ p.34 FCFS

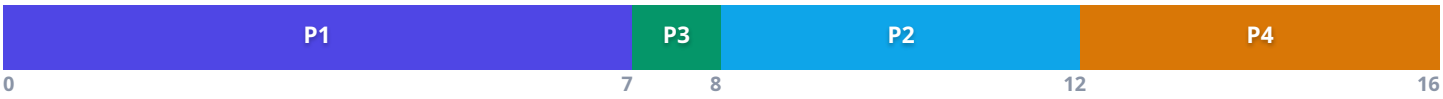


สไลด์ p.35 FCFS — สลับงานสั้นมาก่อน WT=1

C3. SJF — Shortest Job First (non-preemptive) — OPTIMAL

โจทย์ (p.36): P1 (0, 7) · P2 (2, 4) · P3 (4, 1) · P4 (5, 4) — เมื่อ CPU ว่าง เลือก BT สั้นสุดในคิว ณ ตอนนั้น (เท่ากันเลือกมาก่อน)

1. เวลา 0: มีแค่ P1 → รันจนจบที่ 7 (non-preemptive ใครมาทีหลังรอก่อน)
2. เวลา 7: คิว P2(4), P3(1), P4(4) → สั้นสุด P3 → 7→8
3. เวลา 8: P2(4), P4(4) เท่ากัน → มาก่อนชนะ: P2 → 8→12
4. เวลา 12: P4 → 12→16



Process	Completion	TAT	WT = TAT − BT
P1	7	7	7−7 = **0**
P2	12	10	10−4 = **6**
P3	8	4	4−1 = **3**
P4	16	11	11−4 = **7**
เฉลี่ย WT	**—**		**(0+6+3+7)/4 = 4**

📌 SJF คือ optimal

ให้ avg WT ต่ำสุดจากชุดงานหนึ่ง ๆ แต่ใช้จริงยากเพราะ OS ต้อง**ทำนาย**ความยาว CPU burst ถัดไป

C4. SRTF (งานที่เหลือสั้นสุดทำก่อน) — Shortest Remaining Time First (Preemptive SJF) 🔥

กติกา: **งานใหม่มาถึง → เทียบ BT ของงานใหม่ กับ Remaining Time (เวลาที่เหลือ) ของงานที่รันอยู่** — งานใหม่สั้นกว่า = ขัดทันที

1. 0–2: P1 รันไป 2 (เหลือ 5) — P2 มาถึง (BT=4) → เทียบ 4 < 5 → **ขัด P1**
2. 2–4: P2 รันไป 2 (เหลือ 2) — P3 มาถึง (BT=1) → 1 < 2 → **ขัด P2**

3 4–5: P3 จบที่ 5 พอดี + P4 มาถึง (BT=4) → คิว: P1(5), P2(2), P4(4) → รัน P2 จบที่ 7

4 7: เหลือ P1(5), P4(4) → P4 จบที่ 11 → P1 เศษ 5 จบที่ 16

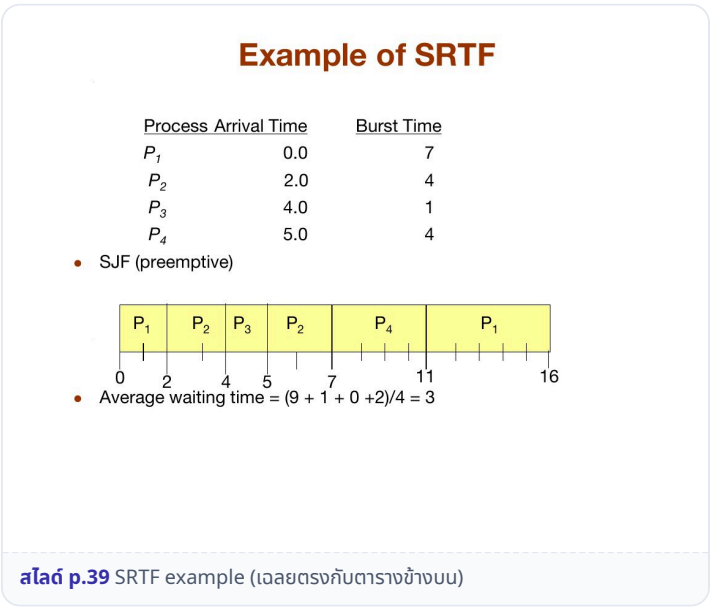


Process	Completion	TAT = C − A	WT = TAT − BT
P1	16	16	16−7 = **9**
P2	7	5	5−4 = **1**
P3	5	1	1−1 = **0**
P4	11	6	6−4 = **2**
เฉลี่ย WT	**—**		**(9+1+0+2)/4 = 3**

> 🎯 **จุดหลอก SRTF**
> เทียบด้วย **Remaining** เสมอ ไม่ใช่ BT ดั้งเดิม! (ตอนเวลา 4 ต้องเทียบ "P2 เหลือ 2" ไม่ใช่ "P2 = 4") · SRTF overhead มากกว่า SJF เพราะ context switch บ่อย



สไลด์ p.38 SRTF นิยาม



สไลด์ p.39 SRTF example (เฉลยตรงกับตารางข้างบน)

C5. Priority (ลำดับสิทธิ์) Scheduling (การจัดคิว)

- CPU ให้งาน priority สูงสุดก่อน (มีทั้ง preemptive/non-preemptive) · SJF = priority ที่ใช้ "predicted next CPU burst" เป็น priority
- **ปัญหา Starvation (การอดตาย (ไม่ได้รันเลย))** (งานสิทธิต่ำไม่ได้รัน) → **แก้ด้วย Aging**: ยิ่งรอนานยิ่งเพิ่ม priority
- **ทิศทางตัวเลข:** Unix-based → **เลขน้อย = สิทธิสูง** / Java Thread → **เลขมาก = สูง** (10 สูงสุด, default 5)

C6. Round Robin (RR) — preemptive, ตอบสนองไว

แต่ละงานรันไม่เกิน **Time Quantum (q)** หมดเวลายังไม่เสร็จ → ถูกตัดไปต่อท้ายคิว · คิวมี n งาน → ไม่มีงานรอเกิน **(n−1)q**

โจทย์ (p.41–42): ข้อมูล FCFS เดิม (P1: 0,24 · P2: 2,3 · P3: 4,3) q = 4



Process	TAT = C − A	WT = TAT − BT
P1	30 − 0 = 30	30 − 24 = **6**
P2	7 − 2 = 5	5 − 3 = **2**
P3	10 − 4 = 6	6 − 3 = **3**
เฉลี่ย WT	**—**	**11/3 ≈ 3.67**

⚖️ ขนาด quantum

q ใหญ่มาก → RR กลายเป็น **FCFS** · q เล็กมาก → **context switch ถี่เกิน overhead โต** (ช่วง switch = pure overhead)

ไทม์ไลน์ RR ทีละจังหวะ

0–4 P1(เหลือ 20) → 4–7 P2 จบ → 7–10 P3 จบ → 10–30 P1 ตัวเดียว รันรอบละ 4 จนครบ 24

Round Robin (RR)

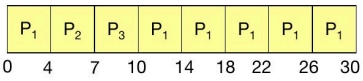
- Every process is allocated a small amount of CPU time, typically ranging from 10 to 100 milliseconds, known as a time quantum.
- Once this time has passed, the process is interrupted and placed at the end of the ready queue
- If the ready queue contains n processes and the time quantum is q, each process is allocated no more than q time units at a time. No process waits more than $(n-1)q$ time units.
- Performance
  - q large ⇒ FIFO
  - q small ⇒ q must be large with respect to context switch, otherwise overhead is too high

สไลด์ p.41 Round Robin

Example of RR with Time Quantum = 4

Process	Arrival Time	Burst Time
P_1	0.0	24
P_2	2.0	3
P_3	4.0	3

- The Gantt chart is:



- Typically, higher average turnaround than SJF, but better *response*

สไลด์ p.42 Example RR q=4 (เฉลยตรงกับตาราง)

C7. Multilevel Queue (คิวหลายชั้น) & Multilevel Feedback Queue (MLFQ)

- **Multilevel Queue**: แยก ready queue หลายคิว (เช่น foreground=RR / background=FCFS) — **งานอยู่คิวไหนตลอดชีวิต ย้ายไม่ได้** · ระหว่างคิว: fixed priority (คิวหน้าจบก่อน — เสี่ยง starvation) หรือแบ่งเวลา (80/20)
- **MLFQ**: **งานย้ายคิวได้** → ทำ aging ได้ จัดกลุ่มตามพฤติกรรมได้

โมเดลตัวอย่าง (p.46–47): Q0 = RR q8ms → ไม่จบ ลง Q1 = RR q16ms → ยังไม่จบ ลง Q2 = FCFS จนจบ · คิวล่างได้รันเมื่อคิวบนว่างหมด

Multilevel Queue Scheduling

- Multilevel Queue Scheduling is a scheduling algorithm that partitions the ready queue into several separate queues.
- Each queue can have its own scheduling algorithm. For Examples
  - foreground – RR
  - background – FCFS
- This method is used when processes can be easily categorized into different classes with different scheduling needs.
- No Movement Between Queues: Processes do not move between queues. Once assigned to a queue, a process stays there.

สไลด์ p.44 Multilevel Queue

Multilevel Feedback Queue Scheduling

- A process can move between the various queues; aging can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters:
  - number of queues
  - scheduling algorithms for each queue
  - method used to determine when to upgrade a process
  - method used to determine when to demote a process
  - method used to determine which queue a process will enter when that process needs service

สไลด์ p.46 MLFQ นิยาม

Example of Multilevel Feedback Queue

- Three queues:
  - Q_0 – RR with time quantum 8 milliseconds
  - Q_1 – RR time quantum 16 milliseconds
  - Q_2 – FCFS
- Scheduling
  - A new job enters queue Q_0 which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue Q_1.
  - At Q_1 job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q_2.

สไลด์ p.47 Example MLFQ 3 คิว

🎯 สรุป 5 จุดพลาดของบทนี้

1) ลบ Arrival ไม่ครบ (เอาเวลาเริ่มรันตอบเป็น WT) 2) WT ใน preemptive ไล่นับมือผิด → ใช้ WT=TAT−BT 3) ตี priority ผิดฝั่ง 4) SRTF ใช้ BT ดิบแทน remaining 5) เลือก quantum ผิด

D. Race Condition & Critical Section (OSNo06 ต้น ๆ)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: Race Condition = ปัญหาที่ 2 process แก้ข้อมูลแชร์พร้อมกันจนผลลัพธ์เพี้ยนตามใครแก้ก่อน-หลัง · Critical Section (ช่วงวิกฤต) = ช่วงโค้ดที่แตะข้อมูลแชร์ ห้ามใครเข้าพร้อมกัน

ความสำคัญกับข้อสอบ: ไล่ timeline การแย่งตัวแปร count (S0–S5), ท่องเงื่อนไข 3 ข้อ (Mutual Exclusion (เข้าถึงได้คนเดียว) / Progress / Bounded Waiting (รอได้จำกัดจำนวนครั้ง)), อธิบาย Peterson's Solution (โซลูชันสำหรับ 2 กระบวนการ) ว่าครบ 3 ข้อยังไง — ออกทั้งถูก/ผิดและไล่ค่า

ใช้จริงในคอมพิวเตอร์: ระบบธนาคารที่โอนเงินพร้อมกัน 2 คน, ตั๋วหนังที่นั่งสุดท้ายถูกซื้อพร้อมกัน, ตัวนับยอดวิว — ถ้าไม่ล็อก ข้อมูลเสียแบบใน timeline เลย

D1. จุดเริ่ม: Bounded Buffer (บัฟเฟอร์ขนาดจำกัด) ยังไม่มี sync (Producer (ฝั่งผู้ผลิตข้อมูล)-Consumer (ฝั่งผู้บริโภคข้อมูล))

Producer-Consumer แชร์ buffer วงแหวนขนาด BUFFER_SIZE ช่อง + ตัวนับ `count` — producer เขียนแล้ว count++ / consumer อ่านแล้ว count−−

```
/* Producer (สไลด์ p.3)                    Consumer (สไลด์ p.4) */
while (true) {                          while (true) {
    while (count == BUFFER_SIZE);           while (count == 0);   // busy wait
    buffer[in] = nextProduced;              nextConsumed = buffer[out];
    in = (in + 1) % BUFFER_SIZE;            out = (out + 1) % BUFFER_SIZE;
    count++;                                count--;
}                                       }
/* ปัญหา: ข้อมูลหาย (producer เขียนทับที่ยังไม่อ่าน) / อ่านซ้ำ — และ count ไม่ atomic */
```

D2. Race Condition — ไล่ S0–S5 ให้เห็นตา 🔥

`count++` ในระดับเครื่องแตกเป็น 3 คำสั่ง (ไม่ atomic!):

```
/* count++: */              /* count--: */
register1 = count;          register2 = count;
register1 = register1 + 1;  register2 = register2 - 1;
count = register1;          count = register2;
```

ไทม์ไลน์จากสไลด์ p.5 (เริ่ม count = 5):

จังหวะ	ใครทำอะไร	ค่าที่เกี่ยวข้อง
S0	producer: register1 = count	r1 = 5
S1	producer: r1 = r1 + 1	r1 = 6
S2	consumer: register2 = count	r2 = **5** ⚠️ (ยังเก่า!)
S3	consumer: r2 = r2 − 1	r2 = 4
S4	producer: count = r1	count = 6
S5	consumer: count = r2	count = **4** ❌ (ควรได้ 5)

💳 อุปมา

เงินในบัญชี 5,000 · คุณโอนเข้า +1,000 แฟนถอน −1,000 พร้อมกัน แอปให้ทั้งคู่เห็นยอด 5,000 พร้อมกันแล้วบันทึกทับ → เหลือ 4,000 เงินหายในอากาศ!

> **📖 นิยามที่ต้องพิมพ์ได้**
> **Race condition**: หลาย process เข้าถึง/จัดการข้อมูลร่วมกันพร้อมกัน แล้ว**ผลลัพธ์ขึ้นกับลำดับการเข้าถึง** · โค้ดช่วงที่แตะข้อมูลแชร์ = **critical section (critical region)**

D3. Critical Section — เงื่อนไข 3 ข้อที่โซลูชันต้องผ่าน 🔥

เงื่อนไข	ความหมาย (พิมพ์ได้ต้องได้)
1. Mutual Exclusion	ถ้า P_i อยู่ใน CS (ช่วงวิกฤต) → **ห้ามมี process อื่นอยู่ใน CS ของมันพร้อมกัน**
2. Progress	ถ้า*ไม่มีใคร*อยู่ใน CS และมีคนอยากเข้า → ต้องเลือกจาก*คนที่อยากเข้าเท่านั้น* และตัดสินไม่ล่าช้าไม่สิ้นสุด (กันคนไม่สนมาขวาง / กันไม่มีใครเข้าเลย)
3. Bounded Waiting	มีขอบเขตว่า process หนึ่งถูก "ตัดหน้า" ได้**จำกัดกี่ครั้ง**หลังยื่นขอ (กัน starvation ของคิว CS)

โครงโค้ด: ENTRY SECTION → CRITICAL SECTION → EXIT SECTION → REMAINDER SECTION (วน while(true)) · สมมติ: แต่ละ process รันด้วยความเร็วไม่เป็นศูนย์ แต่**ไม่มีข้อสมมติเรื่องความเร็วสัมพัทธ์**

D4. Peterson's Solution (2 process)

แชร์ 2 ตัวแปร: `int turn` (ตาใคร) + `boolean flag[2]` (ใครอยากเข้า) — สมมติ LOAD/STORE **atomic**

```
/* โค้ดของ Pᵢ (อีกฝั่งคือ Pⱼ) — สไลด์ p.9–10 */
while (true) {
    flag[i] = TRUE;              // 1. ประกาศว่า "ฉันอยากเข้า!"
    turn = j;                    // 2. ยกสิทธิ์ให้เพื่อนก่อน (ความสุภาพ = หัวใจ)
    while (flag[j] && turn == j); // 3. รอเฉพาะเมื่อ: เพื่อนอยากเข้า "และ" ถึงตาเพื่อน

    CRITICAL SECTION

    flag[i] = FALSE;             // 4. ออกแล้ว — ปล่อยให้คนอื่นเข้าได้
    REMAINDER SECTION
}
```

เงื่อนไข	Peterson ทำไม่ผ่านทำไมยังไง
Mutual Exclusion ✓	ทั้งคู่ตั้ง turn พร้อมกันไม่ได้ — คนที่ตั้ง turn เป็นฝ่ายอื่น**เป็นคนสุดท้าย** ต้องรอก่อน
Progress ✓	เงื่อนไขรอต้องเจอ flag[j] = true จริง — คนที่ไม่สนใจเข้าไม่ขวาง
Bounded Waiting ✓	หลังออกจาก CS อีกฝั่งได้เข้าภายใน 1 รอบ (turn สลับเสมอ)

> **📌 ในโลกจริง**
> Peterson เวิร์คในทฤษฎี แต่ CPU สมัยใหม่ (compiler optimization, out-of-order) อาจพัง — ใช้จริงด้วยฮาร์ดแวร์/ไลบรารี แต่**ออกสอบเยอะ**เพราะสอนหลักคิด

D5. Bakery Algorithm (n process) — จ่ายคิวร้านเบเกอรี่

- ก่อนเข้า CS จ่ายเลขคิว = max(number[0..n−1]) + 1 → คนถือเลข**น้อยสุด**เข้าก่อน · เลขซ้ำเทียบ **PID** มาก่อนชนะ
- เทียบแบบ lexicographical: `(a,b) < (c,d)` ถ้า a < c หรือ (a = c และ b < d) — (เลขคิว, pid)
- แชร์: `boolean choosing[n]`, `int number[n]` (เริ่ม false/0)

```
do {
    choosing[i] = true;                          // กำลังจ่ายเลข (ยังไม่นับคิวจริง)
    number[i] = max(number[0..n-1]) + 1;          // จ่ายเลขใหญ่กว่าใคร ๆ +1 (อาจซ้ำ)
    choosing[i] = false;
    for (j = 0; j < n; j++) {
        while (choosing[j]);                     // รอคนที่กำลังจ่ายเลขจบก่อน
        while ((number[j] != 0) && ((number[j],j) < (number[i],i)));
        // รอถ้า: เพื่อนถือคิวอยู่ และ (เลขน้อยกว่า หรือ เลขเท่าแต่ pid น้อยกว่า)
    }
    CRITICAL SECTION
    number[i] = 0;                               // ออก → ยกเลิกคิวตัวเอง
    REMAINDER SECTION
} while (true);
```

เลขคิวอาจกระโดด (1,2,3,3,3,4) — ไม่เป็นไร lexicographical จัดการเคสซ้ำ · รับประกันครบ 3 เงื่อนไขสำหรับ n process

E. Semaphore · ปัญหาคลาสสิก · Monitor · Java Synchronization (OSNo06 ท้าย ๆ) 🔥

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: Semaphore (ตัวนับควบคุมคิว) = เครื่องมือที่ OS ให้มาจัดคิวเข้าถึงของแชร์อย่างปลอดภัย มีคำสั่ง wait (หยิบสิทธิ์) / signal (คืนสิทธิ์) · Monitor = กล่องควบคุมระดับสูงที่ล็อกให้อัตโนมัติ

ความสำคัญกับข้อสอบ: โค้ด Bounded Buffer / Readers-Writers (ปัญหาคนอ่าน-คนเขียน) / Dining Philosophers (ปัญหานักปราชญ์กินข้าว) ต้องจำค่าเริ่มต้นให้แม่น (mutex=1, empty=n, full=0) + จุดหลอก "สลับลำดับ wait → deadlock (การติดตาย (รอกันตลอดกาล))" — ออกถี่มาก

ใช้จริงในคอมพิวเตอร์: คิวงานเครื่องปริ้นต์, connection pool ของฐานข้อมูล, ปุ่ม like ที่คนกดพร้อมกันหลายพันคน — ระบบจริงทั้งหมดใช้ semaphore (ตัวนับควบคุมสิทธิ์เข้าถึง)/mutex คุมจังหวะ

E1. Semaphore — wait / signal (อุปมาโหลหิน 🪨)

```
wait(S) {                // P — "หยิบหินจากโหล"
    while (S <= 0);      // โหลว่าง → ยืนรอ (busy wait)
    S--;
}
signal(S) {              // V — "คืนหินใส่โหล" แล้วปลุกคนรอ
    S++;
}
```

- **Binary semaphore** = ค่า 0/1 เท่านั้น อีกชื่อคือ **mutex lock (การล็อก)** → ใช้กับ critical section (เข้าได้ทีละคน)
- **Counting semaphore** = ค่า 0..k → ใช้กับ resource (ทรัพยากร) หลายหน่วย (k = จำนวน resource)

🎯 กับดักจาก Quizizz ชุด 3

(1) CS ใช้ **binary** ไม่ใช่ counting (2) Semaphore การันตีได้แค่ **Mutual Exclusion** — progress/bounded waiting ขึ้นกับ implementation

E2. Implement แบบไม่มี busy waiting (waiting queue + block/wakeup)

```
/* แต่ละ semaphore มี waiting queue (แต่ละรายการเก็บ value + pointer รายการถัดไป) */
wait(S) {
    S.value--;                    // หยิบหินก่อนเลย
    if (S.value < 0) {            // ติดลบ = หินไม่พอ มีคนรอแล้ว
        เพิ่มตัวเองเข้า S.list;      // ต่อคิว
        block();                  // หลับ — ไม่เผา CPU
    }
}
signal(S) {
    S.value++;                    // คืนหิน
    if (S.value <= 0) {           // ยังมีคนรออยู่
        หยิบ P จาก S.list;
        wakeup(P);                // ปลุก 1 คนกลับ ready queue
    }
}
/* 💡 ค่า value ติดลบ = มี process รออยู่ |value| ตัว */
```

E3. ปัญหาคลาสสิก 1: Bounded Buffer 🔥 (semaphore 3 ตัว)

Semaphore	เริ่มต้น	ชนิด	หน้าที่
`mutex`	**1**	binary	ล็อก buffer เข้าแตะทีละคน
`empty`	**n**	counting	นับช่องว่าง
`full`	**0**	counting	นับช่องที่มีข้อมูล

```
/* Producer (สไลด์ p.20–21) */
while (true) {
    // produce an item
    wait(empty);   // ขอช่องว่าง
    wait(mutex);   // ล็อก buffer
    // เขียนลง buffer
    signal(mutex); // ปลดล็อก
    signal(full);  // บอกว่ามีของแล้ว
}
```

```
/* Consumer (สไลด์ p.21) */
while (true) {
    wait(full);    // มีของไหม?
    wait(mutex);
    // ดึงของออกจาก buffer
    signal(mutex);
    signal(empty); // คืนช่องว่าง
    // consume item
}
```

🎯 จุดหลอก: สลับ wait → DEADLOCK

ถ้า producer เขียน `wait(mutex); wait(empty);` — buffer เต็ม → producer ครอง mutex ค้างรอ empty แต่ consumer เข้าอ่านไม่ได้เพราะ mutex ถูกครอง → รอกันตาย · **กฎเหล็ก: เช็ค resource (empty/full) ก่อนล็อก (mutex) เสมอ** · จำค่าเริ่มต้นให้แม่น: mutex=1, empty=n, full=0

E4. ปัญหาคลาสสิก 2: Readers-Writers

กติกา: **Reader หลายตัวอ่านพร้อมกันได้** · Writer ทำงานเมื่อไร้ห้ามใครยุ่งเด็ดขาด · แชร์: `wrt=1, mutex=1, readcount=0 (int ธรรมดา)`

```
/* Writer — ง่ายสุด */
wait(wrt);
// เขียนข้อมูล
signal(wrt);
```

```
/* Reader — "ตัวแรกล็อก ตัวสุดท้ายปลด" */
wait(mutex);              // แตะ readcount ทีละคน
readcount++;
if (readcount == 1)       // reader แรกล็อก writer
    wait(wrt);
signal(mutex);
// อ่านข้อมูล (พร้อมกันได้หลายคน)
wait(mutex);
readcount--;
if (readcount == 0)       // reader สุดท้ายปลดล็อก
    signal(wrt);
signal(mutex);
```

🎯 จาก Quizizz 3

semaphore ที่ควบคุมการเพิ่ม/ลดจำนวน reader คือ **mutex** (ครอบ readcount) — **readcount เป็น int ธรรมดา** ไม่ใช่ semaphore · ที่คุม writer คือ wrt

E5. ปัญหาคลาสสิก 3: Dining Philosophers (นักปราชญ์ 5 คน)

ตะเกียบ 5 อัน = **resource** ใช้ semaphore 1 ตัวต่อ 1 ตะเกียบ เริ่มทุกตัว = 1 · กินได้ต้องมีครบ 2 ข้าง (ซ้าย i, ขวา (i+1)%5)

```
/* โค้ดนักปราชญ์ i (สไลด์ p.25–26) */
while (true) {
    wait(chopstick[i]);              // หยิบซ้าย
    wait(chopstick[(i + 1) % 5]);    // หยิบขวา
    // กินข้าว 🍚
    signal(chopstick[i]);            // วางซ้าย
    signal(chopstick[(i + 1) % 5]); // วางขวา
    // คิดปรัชญา
}
```

🎯 โซลูชัน naive นี้พัง

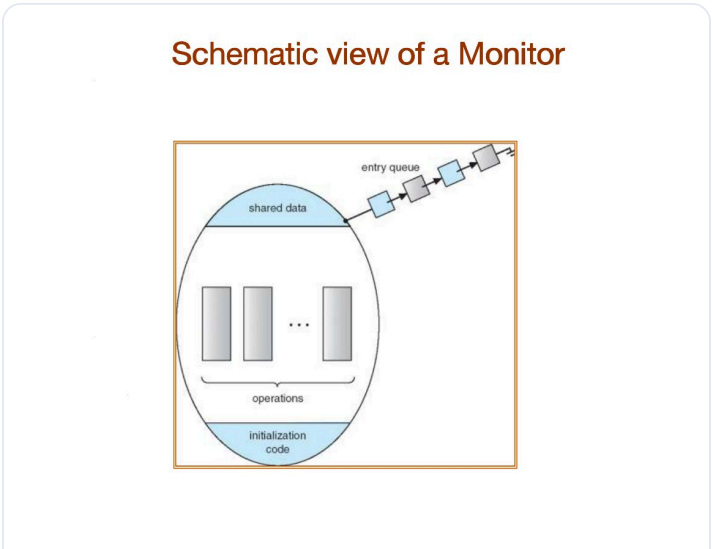
ทุกคน**หิวพร้อมกัน หยิบซ้ายพร้อมกัน** → ไม่เหลือตะเกียบว่าง → ทุกคนรอขวาเป็นวงกลม = **deadlock** (+ ไม่มี bounded waiting → starvation ได้) · แก้: จำกัดจำนวนคนนั่ง ≤ 4 / หยิบพร้อมกันเท่านั้น / ใช้ Monitor (ล่าง)

E6. Monitor — เกราะระดับสูง + Condition (เงื่อนไข) Variables

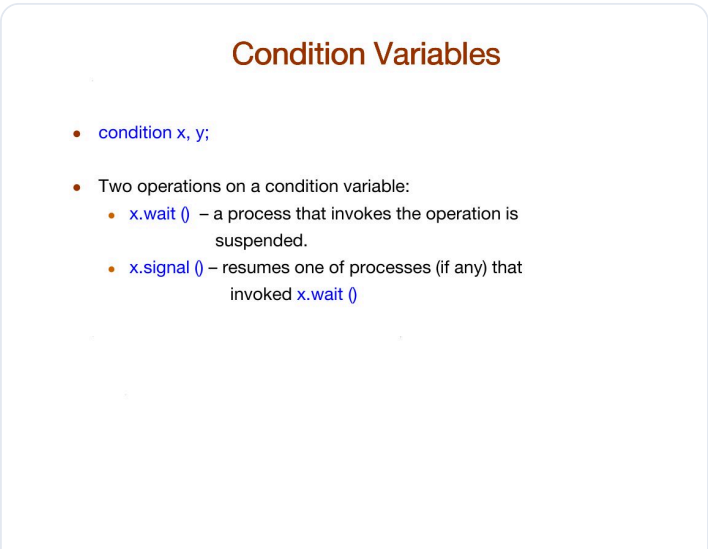
- Monitor = shared data + ฟังก์ชันรวมในก้อนเดียว (คล้าย class) — **รับประกัน mutual exclusion อัตโนมัติ: 1 process ใน monitor ต่อ 1 เวลา** (ลืมล็อกไม่ได้!)
- **Condition variable x:** `x.wait()` = สลัดล็อก หลับไปรอในคิวของ x · `x.signal()` = ปลุก 1 ตัวในคิว x

🎯 ต่างจาก semaphore ยังไง

x.signal() ถ้าไม่มีใครรอ → ***สัญญาณหายไปเฉย ๆ*** (ไม่มี counter เก็บยอด) ต่างจาก semaphore ที่ signal สะสมค่า



สไลด์ p.29 Schematic view of a Monitor: คิว entry → ใน monitor มีได้ 1 คน



สไลด์ p.30 Condition Variables x.wait() / x.signal()

E6.1 แก้ Dining Philosophers ด้วย Monitor (กัน deadlock 100%)

```
monitor DP {
    enum {THINKING, HUNGRY, EATING} state[5];   // สถานะของแต่ละคน
    condition self[5];                          // คิวรอของแต่ละคน

    void pickup(int i) {
        state[i] = HUNGRY;
        test(i);                                // ลองดูว่ากินได้ไหม
        if (state[i] != EATING) self[i].wait;   // ยังไม่ได้ → หลับรอ
    }
    void putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5);                      // เช็คเพื่อนบ้านซ้าย
        test((i + 1) % 5);                      // เช็คเพื่อนบ้านขวา
    }
    void test(int i) {
        // กินได้เมื่อ: เพื่อนซ้ายไม่กิน และ ฉันหิว และ เพื่อนขวาไม่กิน
        if ( (state[(i + 4) % 5] != EATING) &&
             (state[i] == HUNGRY) &&
             (state[(i + 1) % 5] != EATING) ) {
            state[i] = EATING;
            self[i].signal();
        }
    }
}
/* ใช้: dp.pickup(i) → EAT → dp.putdown(i)
   ไม่มีใคร "ถือตะเกียบเดี่ยวค้าง" → ไม่มี deadlock แน่นอน (แต่ starvation ยังเป็นไปได้) */
```

E7. Java Synchronization — synchronized (ล็อกอัตโนมัติ (คีย์เวิร์ด Java)) · wait · notify (ปลุก 1 thread ที่รออยู่)

- ทุก object มี **lock** — เรียก `synchronized` method ต้องได้ lock ก่อน · ตกคิวไปรอใน **Entry Set (คิวรอเข้าล็อก)** · ออกจาก method = คืน lock อัตโนมัติ
- `wait()` ทำ 3 อย่าง: **(1) คืน object lock (2) สถานะ blocked (3) ย้ายเข้า Wait Set (คิวหลับรอเงื่อนไข)**
- `notify()` = **สุ่ม** 1 thread จาก Wait Set → ย้ายไป Entry Set (รอแย่ง lock ใหม่ — **ไม่ได้รับทันที!** และเลือกตัวเองไม่ได้) · `notifyAll()` = ปลุก**ทุกตัว** (ปลอดภัยกว่าเมื่อ wait set มีหลายตัว)
- Java 5+ มี `Semaphore` class จริง: `acquire()` ≈ wait, `release()` ≈ signal

```
/* insert() ด้วย wait/notify (สไลด์ p.41) */
public synchronized void insert(Object item) {
   while (count == BUFFER_SIZE) {
       try { wait(); }                  // เต็ม → คืนล็อก หลับรอ
       catch (InterruptedException e) { }
   }
   ++count;
   buffer[in] = item;
   in = (in + 1) % BUFFER_SIZE;
   notify();                            // ปลุกคนที่รออยู่ 1 ตัว
}
```

```
/* remove() (สไลด์ p.42) */
public synchronized Object remove() {
   Object item;
   while (count == 0) {
       try { wait(); }                  // ว่าง → หลับรอ
       catch (InterruptedException e) { }
   }
   --count;
   item = buffer[out];
   out = (out + 1) % BUFFER_SIZE;
   notify();
   return item;
}
```

🎯 ตารางแยกพวก "wait" ที่หลอกบ่อยสุด

wait ของใคร	ทำอะไร
`wait(S)` ของ Semaphore	หยิบหิน — ลดค่า S
`wait()` ของ Java/Monitor	สลัด lock + หลับใน Wait Set
`wait()` ของ Process (บท A)	พ่อรอลูกจบ เก็บ exit status

แถม: `Thread.yield()` แค่ยอมออกจาก CPU — **ยังถือ lock ค้างอยู่!** (เหมือนล็อกห้องแล้วปีนหน้าต่างหนี)

F. Deadlock + Banker's Algorithm (OSNo07) 🔥

> **📖 รู้จักส่วนนี้ก่อนอ่าน**
>
> **ความหมาย:** Deadlock (การติดตาย) = กลุ่ม process รอทรัพยากรกันเป็นวงกลม ไม่มีใครได้ทำงานต่ออีกเลย · Banker's Algorithm = วิธี "เช็กก่อนให้" ว่าจัดสรรแล้วระบบยังปลอดภัยไหม
>
> **ความสำคัญกับข้อสอบ:** เงื่อนไข 4 ข้อ (ต้องครบพร้อมกัน), อ่าน Resource-Allocation Graph (กราฟแสดงการจัดสรรทรัพยากร), และไล่ Safety Algorithm (อัลกอริทึม (ขั้นตอนวิธี)) หา Safe Sequence (ลำดับการรันที่ปลอดภัย) — เป็นแนวคำนวณหลักของ Final
>
> **ใช้จริงในคอมพิวเตอร์:** ฐานข้อมูลล็อกแถวข้อมูล 2 ธุรกรรมรอกัน, เปิดไฟล์ Excel 2 โปรแกรมล็อกกัน, รถติด 4 แยกไม่มีใครยอมถอย — ระบบจริงเลือก "เมิน + ให้คนรีสตาร์ทเอง" เหมือนที่ OS ทำ

F1. Deadlock คืออะไร + ตัวอย่างรากฐาน

Deadlock = process ≥ 2 ตัว**รอกันไม่สิ้นสุด** เพราะเหตุการณ์ที่จะปลุกนั้นอยู่ในมือกลุ่มผู้รอเอง

```
/* ตัวอย่างโค้ดจากสไลด์ p.4 — S, Q เป็น binary semaphore เริ่ม 1 ทั้งคู่ */
/* P0 */                    /* P1 */
wait(S);    ← S=0 ยึด S     wait(Q);    ← Q=0 ยึด Q
wait(Q);    รอ Q (P1 ถือ)    wait(S);    รอ S (P0 ถือ) → ตายล็อกทั้งคู่
...
signal(S); signal(Q);       signal(Q); signal(S);
```

- **Starvation** = indefinite blocking — process อาจไม่เคยถูกหยิบออกจากคิว semaphore เลย (ต่างจาก deadlock ที่รอกันเป็นวง)
- สไลด์ p.3 สะพานแคบ: รถ = process, ช่วงถนน = resource, แก้โดยให้คันหนึ่ง**ถอย (preempt + rollback)**
- **OS ส่วนใหญ่ (รวม UNIX) ไม่ป้องกัน/ไม่จัดการ deadlock เลย** — ปล่อยให้โปรแกรมเมอร์รับผิดชอบ

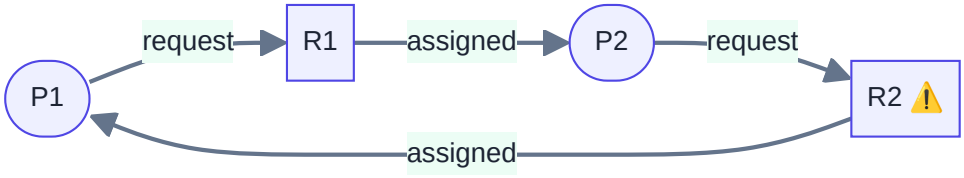
F2. เงื่อนไข 4 ข้อ — ต้องเกิด "พร้อมกัน" จึงเกิด deadlock 🔥

เงื่อนไข	ความหมาย	Prevention ยังไง
1. Mutual Exclusion	ทรัพยากร 1 ชิ้นมีได้ 1 เจ้าของ/เวลา	ทำให้แชร์ได้ — ทำไม่ได้กับทุกชนิด
2. Hold and Wait (ถือของเก่า รอของใหม่)	ถืออย่างน้อย 1 ชิ้น + รอชิ้นอื่นที่คนอื่นถือ	ขอให้**ครบทุกชิ้นก่อนเริ่มรัน** หรือขอเมื่อไม่ถืออะไรเลย → utilization ต่ำ + starvation ได้
3. No Preemption (ยึดคืนกลางคันไม่ได้)	ยึดคืนกลางคันไม่ได้ ปล่อยเมื่อเจ้าของใจดีเท่านั้น	ขอเพิ่มไม่ได้ = ถูกยึดของที่ถือทั้งหมด ไปให้คนรอ แล้วค่อยเริ่มใหม่ → ยุ่งยาก
4. Circular Wait (รอกันเป็นวงกลม)	P0 รอ P1, P1 รอ P2, ... วนกลับมา P0	ให้ทุกทรัพยากรมี**เลขประจำตัว** ขอเรียง**จากน้อยไปมากเท่านั้น** เช่น Disk=4, Printer=11 → ต้องได้ 4 ก่อน 11 ห้ามถือ 11 แล้วขอ 4

> **🎯 จำแน่น**
>
> ต้องครบ **4 ข้อพร้อมกัน** → ตัดข้อเดียวก็ไม่เกิด · Prevention ทั้งหมด "ทฤษฎีดี แต่ปฏิบัติเสียประสิทธิภาพ" อาจารย์สรุปว่าไม่คุ้มใช้

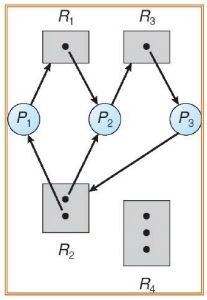
F3. Resource-Allocation Graph (RAG)

- **วงกลม** = Process P_i · **สี่เหลี่ยม** = Resource (ทรัพยากร) type R_j (จุดในสี่เหลี่ยม = จำนวน instance)
- **Request edge** $P_i \rightarrow R_j$: "ขอ" · **Assignment edge** $R_j \rightarrow P_i$: "ได้ถืออยู่"



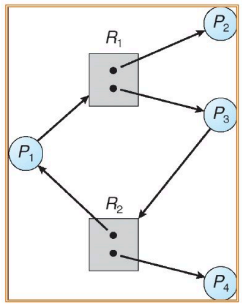
ไม่มี cycle (วงจร (วนกลับมาที่เดิม)) → ไม่เกิด deadlock แน่ (100%)
มี cycle → single instance/type = deadlock แน่ · multiple instances = "อาจเกิด" เท่านั้น
slide p.11: มี cycle แต่ R2 มีหลายชิ้น ชิ้นที่เหลือ (ถือโดย P4 นอกวง) มาช่วยได้ → ไม่ deadlock

Resource Allocation Graph With A Deadlock



สไลด์ p.10 RAG มี deadlock

Graph With A Cycle But No Deadlock



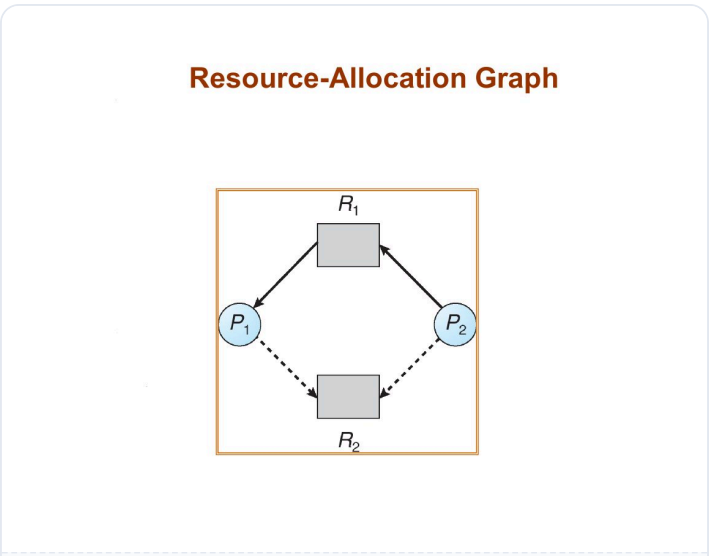
สไลด์ p.11 มี cycle แต่ไม่ deadlock (multiple instance)

F4. กลยุทธ์จัดการ Deadlock 4 แนวทาง

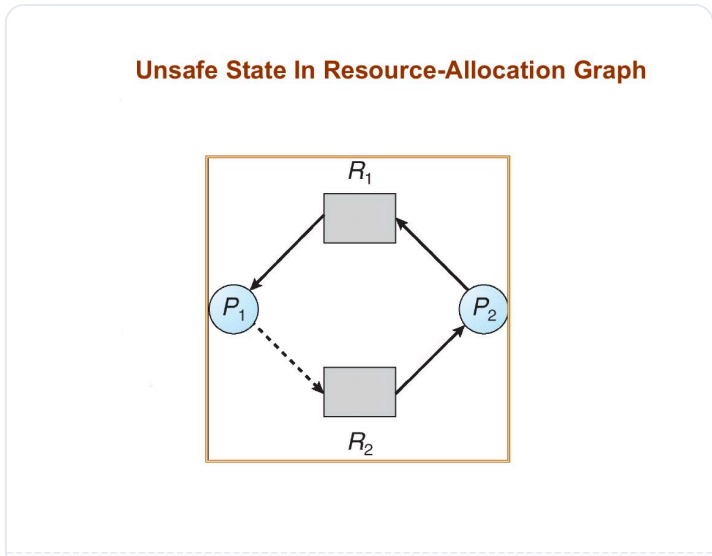
1. **Prevention** — ออกแบบให้เงื่อนไขข้อใดข้อหนึ่งเกิดไม่ได้เลย (ตาราง F2)
2. **Avoidance** — process แจ้ง **Max demand ล่วงหน้า** OS เช็คทุกครั้งก่อนจัดสรรว่ายัง safe → Banker (อัลกอริทึมนายธนาคาร)'s / claim edge
3. **Detection + Recovery** — ปล่อยเกิด สแกนหา แล้วแก้
4. **เมิน (Ostrich)** — ที่ OS จริงใช้กัน

F4.1 Avoidance แบบ single instance — Claim Edge (เส้นประ)

- **Claim edge** $P_i \dashrightarrow R_j$ (เส้นประ) = "อาจขอในอนาคต" · ขอจริง → กลายเป็น request edge · ได้รับ → assignment edge · ปล่อย → กลับเป็น claim edge
- **ให้ทรัพยากรได้เมื่อ:** แปลง request เป็น assignment แล้ว**เกิด cycle ไม่ได้** (มี cycle = unsafe → ไม่ให้ ให้รอ)
- ข้อเสีย: สแกนหา cycle ทุกครั้งที่ขอ = $O(n^2)$ แพง



สไลด์ p.20 Claim edges (เส้นประ)



สไลด์ p.21 Unsafe state: ถ้าให้ P2 ใช้ R2 จะเกิดวง → ไม่ให้

F5. Banker's Algorithm (multiple instances) 🔥🔥

ข้อมูลที่ OS ถือ (n = process, m = resource type): `Available[m]` · `Max[n×m]` · `Allocation[n×m]` · **Need = Max − Allocation**

```
Need[i,j] = Max[i,j] − Allocation[i,j]
```

หัวใจของทุกโจทย์ Banker's — คิด Need ให้ครบทุกช่องก่อนเสมอ

F5.1 Safety Algorithm (ตรวจว่า state ปลอดภัยไหม)

1. Work = Available · Finish[i] = false ทุกตัว
2. หา i ที่ **Finish[i] = false และ $Need_i \le Work$** (เทียบทีละช่องทุกชนิด) — ไม่มีใครผ่าน → ไปขั้น 4
3. สมมติ i รันจนจบแล้วคืนของ: **$Work = Work + Allocation_i$**, Finish[i] = true → กลับขั้น 2
4. ถ้า Finish = true **ทุกตัว** → **SAFE** (มี safe sequence) · ติดตรงไหน = UNSAFE

F5.2 โจทย์เต็มจากสไลด์ p.26–27 (5 process, A=10 B=5 C=7, Available = 3,3,2)

Process	Allocation			Max			Need = Max − Alloc		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

ไล่ Safety (Work เริ่ม = 3,3,2):

ขั้น	เลือกใคร (เหตุผล)	Work หลังคืนของ
1	P1 — Need (1,2,2) ≤ (3,3,2) ✓	(3,3,2)+(2,0,0) = **(5,3,2)**
2	P3 — Need (0,1,1) ≤ (5,3,2) ✓	(5,3,2)+(2,1,1) = **(7,4,3)**
3	P4 — Need (4,3,1) ≤ (7,4,3) ✓	(7,4,3)+(0,0,2) = **(7,4,5)**
4	P2 — Need (6,0,0) ≤ (7,4,5) ✓	(7,4,5)+(3,0,2) = **(10,4,7)**
5	P0 — Need (7,4,3) ≤ (10,4,7) ✓	(10,4,7)+(0,1,0) = (10,5,7) ครบทุกตัว ✅

`Safe sequence = <P1, P3, P4, P2, P0> → SAFE STATE`

มีได้หลายคำตอบ — ตอบลำดับไหนที่ไล่ได้จริงก็ถูก (สไลด์ให้ <P1,P3,P4,P2,P0>)

F5.3 Resource-Request Algorithm — P1 ขอ (1,0,2) ให้ได้ไหม?

1. เช็ค Request ≤ Need: (1,0,2) ≤ (1,2,2) ✓ (ไม่ขอเกินที่ประกาศ)
2. เช็ค Request ≤ Available: (1,0,2) ≤ (3,3,2) ✓ (ของพอ)
3. **สมมติจัดให้ (pretend):** Available = (2,3,0) · P1 Alloc = (3,0,2) · P1 Need = (0,2,0)
4. รัน Safety ใหม่ → ได้ sequence <P1, P3, P4, P0, P2> → ยัง SAFE ✅ → **ให้ไปได้**

คำขออื่นในสไลด์ p.28	ผล	เหตุผล
P4 ขอ (3,3,0)	**ไม่ให้**	Request > Available (3,3,0) > (2,3,0) — ของไม่พอ P4 ต้อง wait
P0 ขอ (0,2,0)	**ไม่ให้**	ผ่าน 2 เช็คแรก แต่ไล่ Safety แล้ว **UNSAFE** (มี deadlock เป็นไปได้) → ให้รอ คืนสถานะเดิม

🟢 วิธีทำในห้องสอบ (เร็ว)

1) เติมตาราง Need = Max − Alloc ทุกช่อง 2) เขียน Work = Available กก.ลงข้าง ๆ 3) วนหา "Need ≤ Work" ใครผ่านให้ไปก่อน แล้ว **บวก Allocation แถวนั้นเข้า Work** 4) จนครบทุกตัว = safe · โจทย์ถาม "ขอได้ไหม" → เช็ค 2 ข้อแล้ว pretend แล้วไล่ safety ใหม่เสมอ

📌 ปัญหาของ Banker's (เคยออก)

ต้องประกาศ Max ล่วงหน้า / จำนวนทรัพยากรคงที่ / จำนวนงานคงที่ / overhead สูง / ใช้ทรัพยากรไม่คุ้มเพราะคิดแย่สุดเสมอ

F6. Detection (ตรวจจับ) & Recovery (กู้)

- **Single instance/type:** ยุบ RAG เป็น **Wait-For Graph (กราฟแสดงว่าใครรอใคร)** — ตัดโหนดทรัพยากรทิ้ง เหลือ $P_i \rightarrow P_j$ ("P_i รอ P_j") มี cycle = deadlock (สแกนทุกระยะ, แพง $O(n^2)$)
- **Multiple instances:** Detection Algorithm ทำงานเหมือน Safety แต่เทียบด้วย **Request** (ของที่ร้องขอค้างจริง) แทน Need · Finish เริ่มต้น: คนที่ Allocation ไม่เป็นศูนย์ = false

ตัวอย่าง Detection จากสไลด์ p.36–37 (A=7, B=2, C=6, Available = 0,0,0):

Process	Allocation			Request		
	A	B	C	A	B	C
P0	0	1	0	0	0	0
P1	2	0	0	2	0	2
P2	3	0	3	0	0	0
P3	2	1	1	1	0	0
P4	0	0	2	0	0	2

ไล่: Work=(0,0,0) → P0 (Request 000 ≤ 000) จบ → Work=(0,1,0) → P2 (000 ≤ 010) จบ → Work=(3,1,3) → P3 จบ → (5,2,4) → P1 จบ → (7,2,4) → P4 จบ → **sequence <P0,P2,P3,P1,P4> ครบ → ไม่มี deadlock**

แต่ถ้า P2 ขอเพิ่ม C หนึ่งชิ้น (Request P2 = 0,0,1): Work=(0,0,0) → P0 จบ → (0,1,0) → ใครผ่าน? P1(2,0,2)✗ P2(0,0,1)✗ P3(1,0,0)✗ P4(0,0,2)✗ → ตัน → **deadlock: P1, P2, P3, P4**

Recovery 2 แนวทาง

- **Process Termination:** จบทุกตัวที่ติด (เสียงานหมด) หรือจบทีละตัวจนวงแตก — เลือกจาก: priority, รันมานานแค่ไหน, ใช้ทรัพยากรอะไรไปแล้ว, ต้องการอีกเท่าไร, ต้องจบกี่ตัว, interactive/batch?
- **Resource Preemption (การยึดทรัพยากรกลางคัน):** เลือกเหยื่อ (victim) → **Rollback** ไป safe state → เริ่มใหม่ · ระวัง **Starvation** — ใส่ "จำนวน rollback ที่โดนมาแล้ว" ในตัวแปรต้นทุน

G. Real Memory Management (OSNo08 ต้น)

> 📖 **รู้จักส่วนนี้ก่อนอ่าน**
>
> **ความหมาย:** การจัดสรร RAM ให้ process ตอนโหลด (ยุคก่อนมี virtual memory) — จัดแบบช่องคงที่, ช่องแปรผัน, หรือแบ่งตามส่วน (Segmentation (การแบ่งหน่วยความจำตามส่วน)) พร้อมกลยุทธ์เลือกหลุม First/Best/Worst-fit (หยิบหลุมใหญ่ที่สุด)
>
> **ความสำคัญกับข้อสอบ:** โจทย์ first/best/worst-fit คำนวณเร็วได้คะแนนง่าย + Segmentation (เช็ค offset (ตำแหน่งเลื่อนภายในเพจ) < limit ก่อน แล้วค่อย base + offset) — และการแยก internal/external (ภายนอก (กระจายเป็นหลุม)) fragmentation (การแตกเป็นเศษพื้นที่) ให้ถูกตัว
>
> **ใช้จริงในคอมพิวเตอร์:** เป็นต้นแบบ memory allocator (malloc/free) ในทุกภาษา · ปัญหา fragmentation ยังพบจริงในระบบฝังตัว (embedded) และการจัดการหน่วยความจำ GPU

G1. Logical (เชิงตรรกะ (มองจากโปรแกรม)) vs Physical Address (ที่อยู่จริงใน RAM) + MMU

	Logical (Virtual) Address	Physical Address
สร้างโดย	CPU ปล่อยขณะรัน	ตำแหน่งจริงที่ memory unit มองเห็น
จุดอ้างอิง	โปรแกรมเริ่มที่ byte 0 เสมอ	เทียบจุดเริ่ม RAM จริง

MMU = ฮาร์ดแวร์แปลง logical (เชิงตรรกะ (มองจากโปรแกรม)) → physical (เชิงกายภาพ (ตำแหน่งจริง)) แบบง่ายสุดคือ**บวก Relocation (Base) Register**

`Physical Address = Relocation Register (รีจิสเตอร์เก็บตำแหน่งฐาน) + Logical Address (ที่อยู่เชิงตรรกะ (มองจากโปรแกรม))`

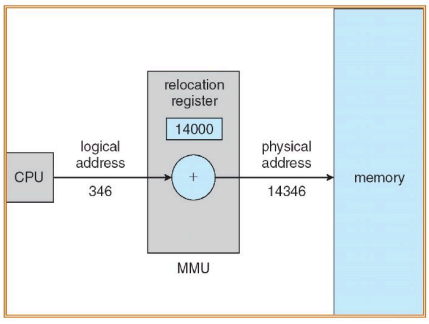
ตัวอย่างวิดีโอ: โปรแกรมโหลดที่ 14,000 · CPU ขอ byte 346 → PA = 14,000 + 346 = 14,346

Logical vs. Physical Address Space

- **Logical address** – generated by the CPU; also referred to as **virtual address**
- **Physical address** – address seen by the memory unit

สไลด์ p.3 Logical vs Physical (เชิงกายภาพ (ตำแหน่งจริง))

Dynamic relocation using a relocation register



สไลด์ p.5 Dynamic relocation ด้วย relocation register

G2. Static vs Dynamic Linking (มีตัวเลขให้คำนวณ)

- **Static linking**: ก๊อปโค้ด library รวมใน executable ตั้งแต่ compile — lib ซ้ำทุกโปรแกรม
- **Dynamic linking**: เลื่อนไป link (ลิงก์ (ทางเชื่อม)) ตอน run — ใส่แค่ **stub** (โค้ดจิ๋วหา library / แทนตัวเองด้วย address routine) → library โหลดครั้งเดียว แชร์กันทั้งระบบ (shared libraries)

`ตัวอย่างวิดีโอ: A(10MB) + B(20MB) + Lib(20MB)`
`Static: A=30MB, B=40MB → RAM 30+40 = 70MB (lib ซ้ำ 2 ชุด)`
`Dynamic: A=10+2, B=20+2, lib 1 ชุด → 12+20+22 = 54MB ประหยัด 16MB`

G3. Fixed Partition — Internal Fragmentation (เศษภายในช่องที่จองแล้ว)

- แบ่ง RAM ส่วน user เป็นช่องคงที่ล่วงหน้า → **มีกี่ช่องรับ process พร้อมกันได้เท่านั้น** · ป้องกันด้วย limit register (ขนาด) + relocation register (ตำแหน่ง): เช็ค LA < Limit ผ่านแล้วค่อย + Base
- **ตัวอย่างวิดีโอ:** Partitions P1=100K, P2=25K, P3=25K, P4=50K · Job1(30K)→P1 (เหลือ 70K ใช้ไม่ได้ = **internal frag**) · Job2(50K)→P4 พอดี · **Job3(30K) ต้องรอ!** เพราะ P2,P3 มีแค่ 25K แม้ว่างรวม 120K · Job4(25K)→P2 พอดี

🎯 คู่แฝด fragmentation

Fixed = Internal (เศษในช่องจองแล้ว ใครใช้ต่อไม่ได้) · **Variable + Segmentation = External** (เศษกระจายนอก รวมว่างเยอะแต่ช่องเดียวไม่พอ) · **Paging = ไม่มี External แต่มี Internal** ที่เพจสุดท้าย — ออกถูก/ผิดวนไป

G4. Variable Partition — External Fragmentation (เศษกระจายนอก (หลุมเล็ก ๆ)) + วิธีรักษา

จัดสรรตามขนาดจริง ตอนแรกไม่มีของเสีย แต่งานจบทิ้งช่องว่างกระจาย · **ตัวอย่าง:** หลุมว่าง 5+2+1 = 8MB แต่ขอ 7MB โหลดไม่ได้ (ช่องเดียวใหญ่สุด 5MB)

วิธีรักษา	ทำอะไร	ข้อจำกัด
Coalescing (การรวมหลุมว่างที่ติดกัน)	รวมหลุมที่**ติดกัน**เป็นหลุมใหญ่ 1 หลุม	หลุมไม่ติดกันก็ช่วยไม่ได้ — มักไม่พอ
Compaction (การอัดข้อมูลให้ติดกัน)	ลาก process มา**อัดติดกัน** รวมว่างปลายเดียว	overhead แพง และทำได้**เฉพาะ relocation แบบ dynamic (ตอน execution time)**

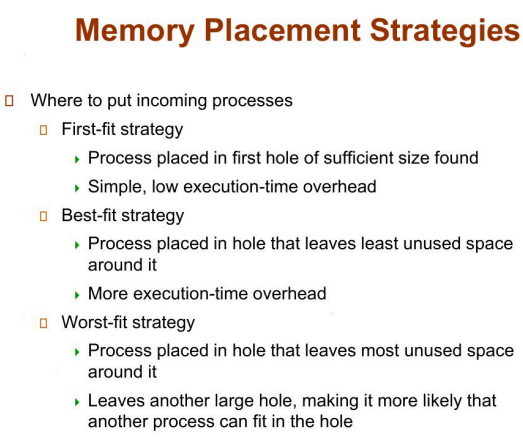
G5. First-fit (หยิบหลุมแรกที่พอ) · Best-fit (หยิบหลุมที่แน่นที่สุด) · Worst-fit 🔥 (ตัวอย่างจากวิดีโอ)

โจทย์ (สไลด์ p.21–24): หลุมเรียงตามลำดับใน RAM: **16, 14, 5, 30 MB** · process ใหม่ต้องการ **13 MB**

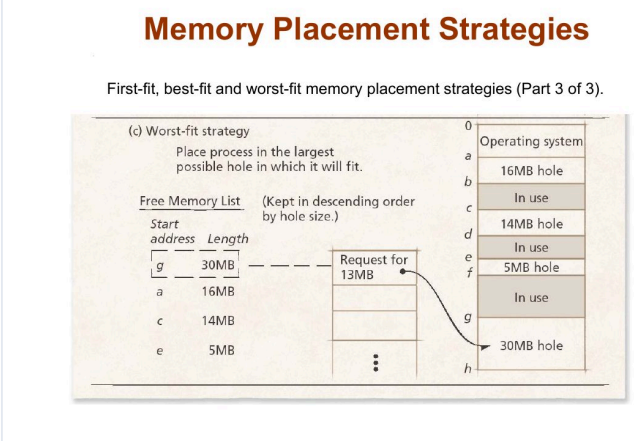
กลยุทธ์	หลักคิด	ลงหลุม	เศษเหลือ	จุดเด่น/ด้อย
First-fit	ไล่จากซ้าย เจอหลุมแรกที่ ≥ ขนาด เอาทันที	**16**	3	เร็วสุด (ไม่สแกนครบ) แต่เปลืองพื้นที่
Best-fit	สแกนครบ เลือกหลุมที่ "ใหญ่พอ + เหลือน้อยสุด"	**14**	1	คุ้มสุด แต่ช้า + ทิ้งเศษจิ๋วใช้ต่อไม่ได้
Worst-fit	สแกนครบ เลือกหลุมใหญ่สุด	**30**	17	ทิ้งชิ้นใหญ่ให้งานต่อไป (โดยรวมมักแพ้ best-fit)

🟢 เทคนิค 30 วินาที

เขียนเลขหลุมตามลำดับที่โจทย์ให้ (ห้ามจัดใหม่!) · First = หยุดหลุมแรกที่พอ · Best = เทียบ (หลุม−ขนาด) เลือกค่าน้อยสุด · Worst = หลุมใหญ่สุด · ถ้าโจทย์ให้จัดหลายงานต่อกัน อย่าลืมหักขนาดหลุมที่ใช้ไปแล้วด้วย



สไลด์ p.21 Memory Placement Strategies



สไลด์ p.24 ผลการจัดทั้ง 3 กลยุทธ์ (ตรงกับตารางข้างบน)

G6. Segmentation 🔥 — แบ่งตามสายตาโปรแกรมเมอร์

- แบ่งเป็น logical unit ขนาดไม่เท่ากัน: main, function, object, stack, global variables · Logical address = คู่ `<segment s, offset d>`
- Segment table แต่ละแถวมี **Base** (จุดเริ่มใน RAM) + **Limit** (ความยาว) · STBR ชี้ตาราง, STLR = จำนวน segment (s ใช้ได้ถ้า s < STLR)

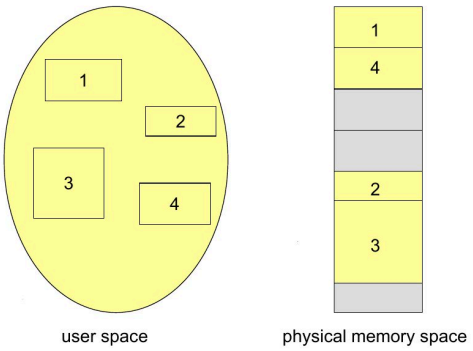
ขั้นแรก: เช็ค d < Limit — ถ้า d ≥ Limit → TRAP (error!) ทันที
ขั้นต่อมา: Physical Address = Base + Offset (ตำแหน่งเลื่อนภายในเพจ)

ตัวอย่างจากสไลด์ p.50: S1 (base 6300, limit 500) · S2 (base 4300, limit 400)

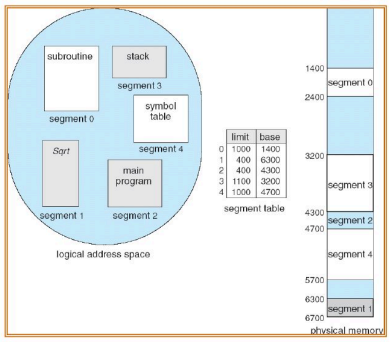
Logical	เช็ค	ผล
<1, 300>	300 < 500 ✓ → PA = 6300 + 300	**6600** ✓
<2, 400>	400 < 400 ✗ (400 ไม่ได้ "น้อยกว่า" 400)	**Trap / Error** ❌

🎯 จุดหลอก

เช็ค **ก่อนเสมอ แล้วค่อยบวก** — เคร่งครัดเรื่อง < ไม่ใช่ ≤ · Segmentation ยังเจ็บกับ **external fragmentation** (แต่ละ segment ต้องวางติดกันเป็นผืน) · สูตรอย่างสลับกับ paging (การแบ่งหน่วยความจำเป็นหน้า): Segment/Base = **Base + offset** / Paging = **frame (เฟรม (ก้อน RAM)) × size (ขนาด) + offset**



สไลด์ p.47 Logical view (user space) vs physical memory



สไลด์ p.50 Example of Segmentation (ตัวเลขตรงกับตาราง)

H. Paging & Address Translation (OSNo08 ท้าย) 🔥🔥

> **📖 รู้จักส่วนนี้ก่อนอ่าน — รีวิวรุ่นพี่: "คนส่วนมากคำนวณ Paging กันไม่ทัน"**
>
> **ความหมาย:** Paging (การแบ่งหน่วยความจำเป็นหน้าเท่า ๆ กัน) = หั่น RAM เป็น frame หั่นโปรแกรมเป็น page (ขนาดเท่ากัน) แล้วใช้ Page Table (ตารางจับคู่เพจ↔เฟรม) จับคู่ — "คำนวณ Paging" คือการแปลงที่อยู่ที่โปรแกรมเห็น (Logical) ให้เป็นที่อยู่จริงใน RAM (Physical)
>
> **ความสำคัญกับข้อสอบ:** แปลง address ทีละบิต, TLB (แคชตารางเพจความเร็วสูง) + สูตร EAT (เวลาเข้าถึงเฉลี่ย), Two-level paging — เป็นแนวคำนวณที่ตัดสินว่า "ทันสอบหรือไม่ทัน" ต้องฝึกจนข้อละไม่เกิน 1 นาที
>
> **ใช้จริงในคอมพิวเตอร์:** CPU ทุกตัวสมัยใหม่ (x86, ARM ในมือถือ) + OS ทุกตัว (Windows/Linux/macOS/Android) ใช้ paging จริง 100% — แอปแต่ละตัวเห็น "RAM ของตัวเอง" ก็เพราะกลไกนี้

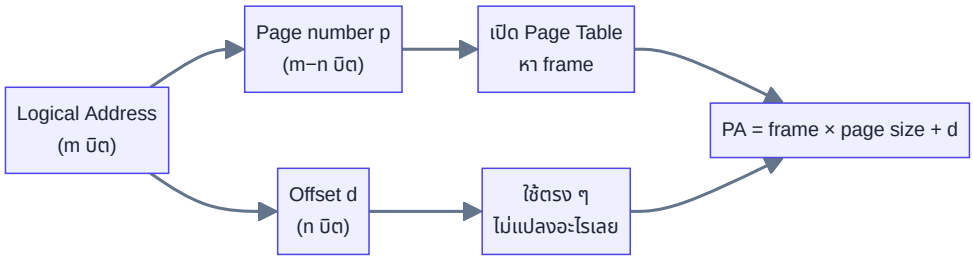
H1. แนวคิด Paging

- **Paging = จัดสรรแบบ non-contiguous** (ไม่ต้องติดกัน) — ตัด external fragmentation ขาด
- **Frame (เฟรม (ก้อน RAM))**: แบ่ง RAM เป็นบล็อกขนาดคงที่ ขนาดเป็นกำลังสอง 2^n bytes (512–8,192)
- **Page (หน้า)**: แบ่งโปรแกรมเป็นบล็อก**ขนาดเท่า frame เป๊ะ** · รันโปรแกรม n เพจ = หา frame ว่าง n อันกระจายวาง
- **Page table**: จับคู่ "เพจ i → frame ไหน" ของแต่ละ process

> **🎯 เช็คความเข้าใจ**
>
> Paging **ไม่มี External frag** แต่มี **Internal frag** — เกิดเฉพาะ**เพจสุดท้าย**ถ้าขนาดโปรแกรมไม่ลงตัว

H2. โครงสร้าง Logical Address — บิต p กับ d 🔥

Address ที่ CPU ปล่อย (ยาว **m บิต**) ซอยเป็น 2 ส่วนโดยฮาร์ดแวร์:



`Page size = 2ⁿ bytes → บิต offset = n บิต · บิต page number (หมายเลขเพจ) = m − n บิต → เพจได้สูงสุด 2^(m−n) เพจ`

ตัวอย่างวิดีโอ: m = 5 บิต (2^5 = 32 bytes), page = 4 bytes (2^2) → offset 2 บิต, page# 3 บิต → 8 เพจ (0–7 นับจากศูนย์!)

> **🪑 อุปมาอาจารย์**
>
> เก้าอี้ 32 ตัว จัด 8 ห้อง ห้องละ 4 — อ้างอิงว่า "เก้าอี้ตัวที่ 3 ในห้องเบอร์ 1" = (Page 1, Offset 3) สองเลขนี้คือทั้งชีวิตของ paging

H3. แปลง Logical → Physical ทีละขั้น 🔥

`Physical Address = (Frame Number × Page Size (ขนาดของเพจ)) + Offset`

เปิดตารางด้วย p → ได้ frame → คูณขนาด frame = จุดเริ่ม → บวก offset

โจทย์จากสไลด์ p.32 (Paging Example): page size (ขนาดของเพจ) = 4 bytes · Page table: 0→5, 1→4, 2→1, 3→7

หาอะไร	Logical	ไล่ทีละขั้น	PA
ตัวอักษร 'C'	(p=0, d=2)	เพจ 0 → frame 5 → (5 × 4) + 2 = 20 + 2	**22**
ตัวอักษร 'K'	(p=2, d=2)	เพจ 2 → frame 1 → (1 × 4) + 2 = 4 + 2	**6**

🎯 3 จุดพลาดคลาสสิก

1) **บวก frame+offset ตรง ๆ** (5+2=7 ❌) ต้องคูณขนาดก่อน 2) นับเพจเริ่ม 1 → ต้องเริ่ม **Page 0** 3) ปัดจำนวนเพจลง — 42 bytes / เพจ 4 = 10.5 → **ปัดขึ้น 11 เพจ** (10 เพจเก็บได้ 40 bytes ข้อมูลขาด!) เพจสุดท้ายใช้ 2 bytes → internal frag = 4−2 = **2 bytes**

Address Translation Scheme

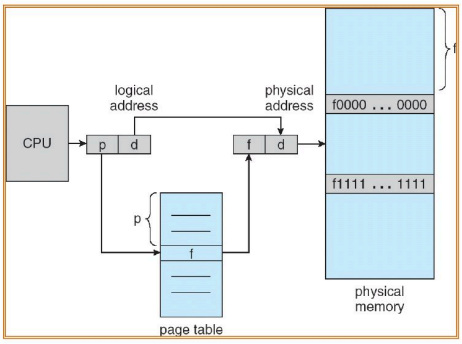
- Address (m bits) generated by CPU is divided into:
  - **Page number (*p*)** – used as an index into a *page table* which contains base address of each page in physical memory
  - **Page offset (d)** – combined with base address to define the physical memory address that is sent to the memory unit

page number	page offset
p	*d*
m - n	*n*

- For given logical address space 2^m *and page size* 2^n

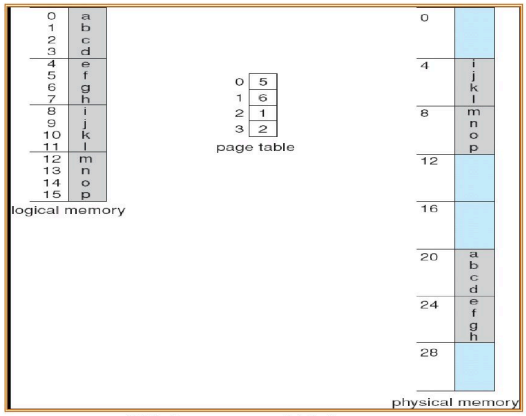
สไลด์ p.29 Address Translation Scheme (p | d)

Paging Hardware



สไลด์ p.30 Paging Hardware: p ไปตารางได้ frame + d ต่อกันเป็น PA

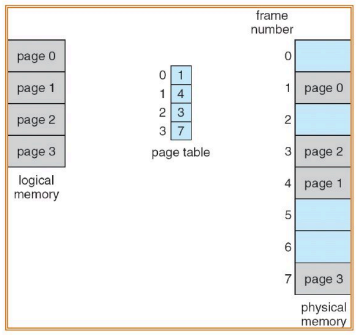
Paging Example



32-byte memory and 4-byte pages

สไลด์ p.32 Paging Example — หน่วยความจำ 32 bytes, เพจ 4 bytes (โจทย์ 'C' และ 'K' อยู่นี่)

Paging Model of Logical and Physical Memory



สไลด์ p.31 Paging Model: เพจกระจาย frame ไม่ติดกันก็ได้

H4. TLB + Effective Access Time 🔥

- Page table อยู่ใน RAM → อ่านข้อมูล 1 ครั้ง = ไป RAM **2 ครั้ง** (1: อ่านตารางหา frame, 2: อ่านข้อมูลจริง)
- **TLB** = แคชฮาร์ดแวร์ความเร็วสูง เก็บคู่ (page # → frame #) ล่าสุด ค้นแบบขนาน (associative memory)

กรณี	ไป RAM	เวลา (memory = 1 μs, TLB search = ε)
TLB Hit (โอกาส α)	1 ครั้ง	ε + 1 = 1 + ε
TLB Miss (โอกาส 1−α)	2 ครั้ง	ε + 1 + 1 = 2 + ε

EAT = α(1 + ε) + (1 − α)(2 + ε) = 2 + ε − α

โจทย์วิดีโอ: ε = 0.1 µs, α = 80% → (0.8×1.1) + (0.2×2.1) = 0.88 + 0.42 = 1.3 µs (สูตรลัด: 2 + 0.1 − 0.8 = 1.3 ✓)

Associative Memory

- Associative memory – parallel search

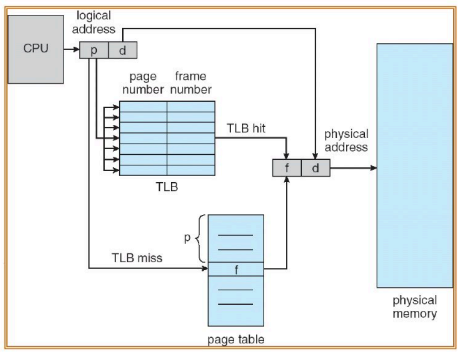
Page #	Frame #

Address translation (p, d)

- If p is in associative register, get frame # out
- Otherwise get frame # from page table in memory

สไลด์ p.35 Associative Memory (TLB): ค้นขนาน

Paging Hardware With TLB



สไลด์ p.36 Paging Hardware With TLB

Effective Access Time

- Associative Lookup = ε time unit
- Assume memory cycle time is 1 microsecond
- Hit ratio – percentage of times that a page number is found in the associative registers; ratio related to number of associative registers
- Hit ratio = α
- **Effective Access Time** (EAT)

$$EAT = (1 + \varepsilon)\,\alpha + (2 + \varepsilon)(1 - \alpha)$$
$$= 2 + \varepsilon - \alpha$$

สไลด์ p.37 Effective Access Time สูตร (ตรงกับข้างบน)

H5. Protection — Valid / Invalid bit

- แต่ละแถว page table (ตารางจับคู่เพจ ↔ เฟรม) มี **valid–invalid bit**: **v** = เพจนี้อยู่ใน address space ของ process (ใช้ได้) · **i** = ห้ามอ้างอิง
- **PTBR** ชี้ตำแหน่ง page table ใน RAM · **PTLR** = ความยาวตาราง (เช็คเบื้องต้นว่าไม่อ้างเพจเกินขอบเขต)

🎯 จุดหลอก: valid ดู "ทั้งเพจ" ไม่ได้ดู "ทีละไบต์"

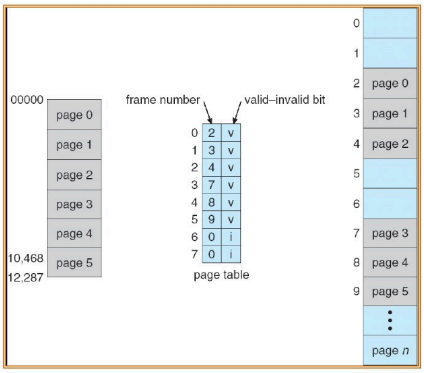
เพจสุดท้าย valid = v → ระบบยอมให้อ้าง offset *ใด ๆ* ในเพจนั้น (แม้เกินขนาดโปรแกรมจริง) — นี่คือเหตุผลที่ internal fragmentation ยังอยู่รอดใน paging

Memory Protection

- Memory protection implemented by associating protection bit with each frame
- **Valid-invalid** bit attached to each entry in the page table:
  - “valid” indicates that the associated page is in the process’ logical address space, and is thus a legal page
  - “invalid” indicates that the page is not in the process’ logical address space

สไลด์ p.38 Memory Protection

Valid (v) or Invalid (i) Bit In A Page Table

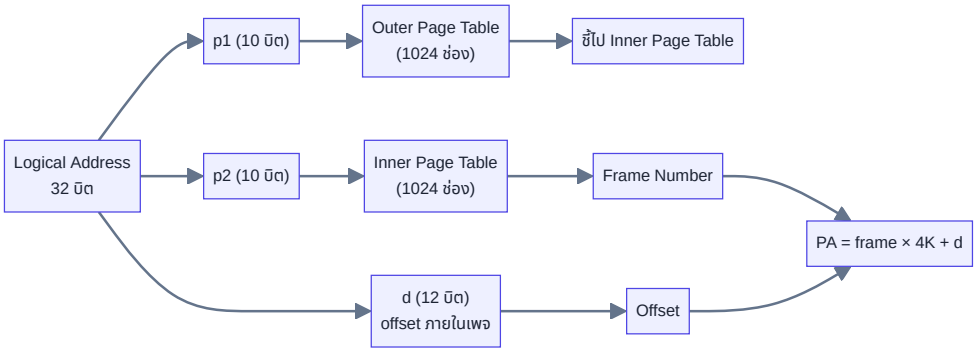


สไลด์ p.39 Valid (v) / Invalid (i) Bit

H6. Two-Level Paging — ทำไม + คำนวณ 🔥

ปัญหา single-level ในระบบ 32-bit: page 4KB (offset 12 บิต) → page# 20 บิต → 2^{20} = 1,048,576 ช่อง × 4B = **4MB ต่อ process ทุกตัว!** แม้โปรแกรมเล็กแค่ 1MB (ตารางต้องอยู่ต่อเนื่องด้วย)

ทางแก้: ซอยบิต 32 ตัวเป็น **p_1 = 10 | p_2 = 10 | d = 12** — p_1 เปิด outer table หา inner table, p_2 เปิด inner table หา frame



โจทย์ (จากวิดีโอ)	จำนวนเพจ	Inner tables ที่ต้องสร้าง	พื้นที่ page table รวม
โปรแกรม 1 MB	1024KB ÷ 4KB = **256 เพจ**	256 ≤ 1024 → **1 ตาราง**	outer 4KB + inner 4KB = **8 KB** (แทน 4MB!)
โปรแกรม 8 MB	8192KB ÷ 4KB = **2048 เพจ**	2048 ÷ 1024 = **2 ตาราง**	4KB + 2×4KB = **12 KB**

🟢 สูตรรวมทั้งบท

บิต offset = $\log_2$(page size) · บิต page# = บิต address − บิต offset · เพจ = ceil(ขนาดโปรแกรม ÷ page size) · PA = frame × pageSize + offset · EAT = 2 + ε − α · inner tables = ceil(จำนวนเพจ ÷ 1024)

Hierarchical Page Tables

- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table

สไลด์ p.42 Hierarchical Page Tables

Three-level Paging Scheme

outer page	inner page	offset
p_1	p_2	d
42	10	12

2nd outer page	outer page	inner page	offset
p_1	p_2	p_3	d
32	10	10	12

สไลด์ p.44 Three-level Paging Scheme (x86 มีจริง)

หมายเหตุ: ยังมี Hashed Page Table และ Inverted Page Table — รู้จักชื่อพอ (inverted: 1 แถวต่อ 1 frame จริง ค้นด้วย (pid, page))

I. Virtual Memory & Page Replacement (OSNo09) 🔥🔥

📖 รู้จักส่วนนี้ก่อนอ่าน

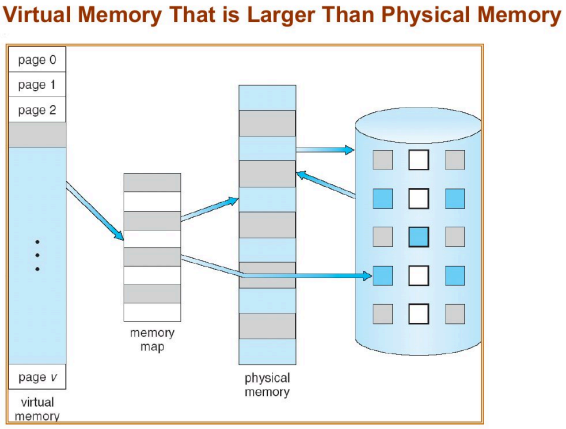
ความหมาย: Virtual Memory (หน่วยความจำเสมือน) = โปรแกรมไม่ต้องอยู่ใน RAM ครบทั้งตัว โหลดเฉพาะเพจที่กำลังใช้ (demand paging) ทำให้ RAM จริงเหมือนใหญ่กว่าจริง · Page Replacement = เมื่อ RAM เต็ม ต้องเลือกเพจ "ตัวปลอด" ออก

ความสำคัญกับข้อสอบ: นับ page fault (เพจยังไม่อยู่ใน RAM) ทั้ง FIFO (เข้าก่อน-ออกก่อน) / Optimal (ดีที่สุดเชิงทฤษฎี) / LRU (รู้ผล 15/9/12 ของ string หลัก), Belady's Anomaly (ปรากฏการณ์เพิ่ม RAM แล้ว fault เพิ่ม), Thrashing (สลับเพจหนักจนระบบแย่) — เป็นแนวคำนวณหลัก อีกแนวของ Final

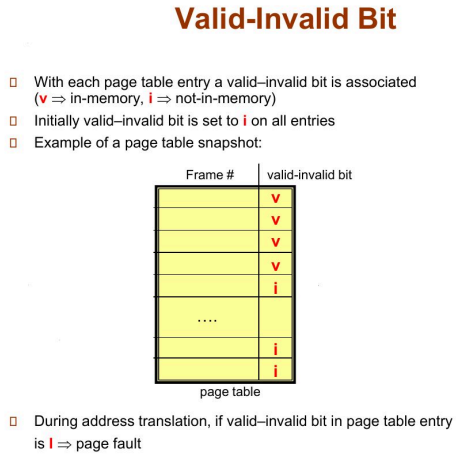
ใช้จริงในคอมพิวเตอร์: เปิดเกมใหญ่กว่า RAM เครื่องได้, สลับแอปเร็วโดยไม่โหลดใหม่, ขนาด "หน่วยความจำเสมือน" ที่เห็นใน Task Manager คือ กลไกนี้ — ข้อมูลที่ไม่ได้ใช้ถูกส่งไปพักบน disk

I1. Demand Paging (โหลดเพจเฉพาะเมื่อถูกใช้) — ขอเมื่อไรค่อยเอามา

- **Virtual memory** = แยก logical memory ออกจาก physical → address space ใหญ่กว่า RAM จริงก็ได้ · implement ผ่าน **demand paging** หรือ demand segmentation
- **Demand paging**: โหลดเพจเฉพาะเมื่อถูกเรียกใช้ — หลักขี้เกียจของ **lazy swapper** (ตัวทำงานกับเพจเรียก **pager**) · ผลดี: I/O น้อย, RAM น้อย, ตอบเร็ว, รองรับ user มาก
- เพจที่ยังไม่โหลด → bit = **i (invalid)** → แปลง address เจอ i → **Page Fault (เพจยังไม่อยู่ใน RAM)** — **invalid ที่นี่ส่วนใหญ่ไม่ใช่ error** แค่ "ยังไม่มา"

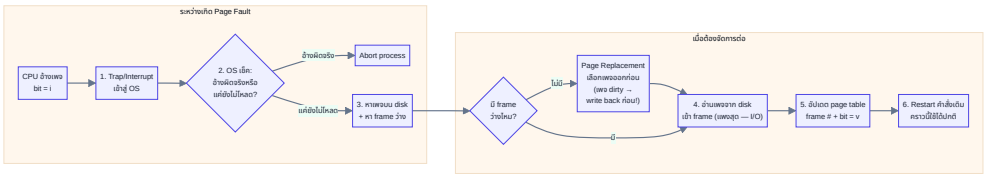


สไลด์ p.4 Virtual Memory ใหญ่กว่า Physical



สไลด์ p.6 Valid–Invalid Bit ใน page table

I2. Page Fault — ขั้นตอนจัดการ 6 ขั้น (เรียงได้ = ได้คะแนน)



⚠️ Dirty page

เพจที่ถูกแก้ไข (dirty (เพจที่ถูกแก้ไขข้อมูลแล้ว)/modified = 1) ต้อง**เขียนกลับ disk ก่อน** (write back (เขียนกลับลงดิสก์ก่อน)) ค่อยแทนที่ — เพจ clean (เพจที่ยังไม่ถูกแก้ไข (ดิสก์มีสำเนาแล้ว)) ทับได้เลยเพราะ disk มีสำเนา

I3. Page Replacement — นับ fault 3 อัลกอริทึม 🔥🔥

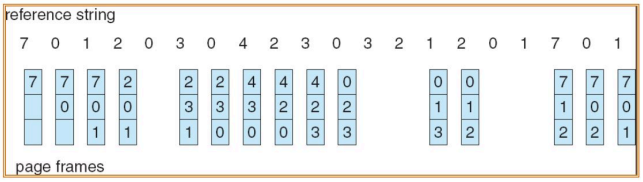
โจทย์หลัก (สไลด์ใช้ string เดียวกัน, 3 frames (เฟรมหลายก้อน) เริ่มว่าง):

Reference String (ลำดับการอ้างเพจ): 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

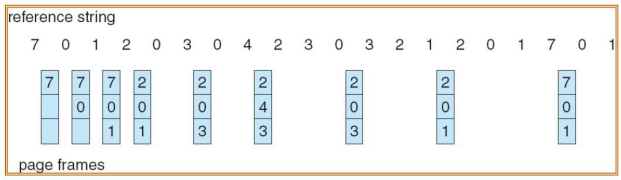
อัลกอริทึม	มองไหน	หลักเลือก victim	Faults (3 frames)	ใช้จริง?
FIFO	คิวเข้า	เข้ามาก่อน ออกก่อน (ไม่สนว่าใครฮิต)	**15**	ง่ายแต่โง่ · เจอ Belady
Optimal (OPT)	อนาคต 👁️	ตัวที่**จะไม่ถูกใช้อีกนานสุด**ในอนาคต	**9**	**ใช้จริงไม่ได้** (รู้อนาคตไม่ได้) — ไว้เทียบ benchmark
LRU	อดีต 🔙	ตัวที่**ไม่ได้ใช้มานานสุด**	**12**	ใช้จริงได้ (เก็บประวัติย้อนหลัง) ดีกว่า FIFO

👕 ตู้เสื้อผ้า 3 ชั้น

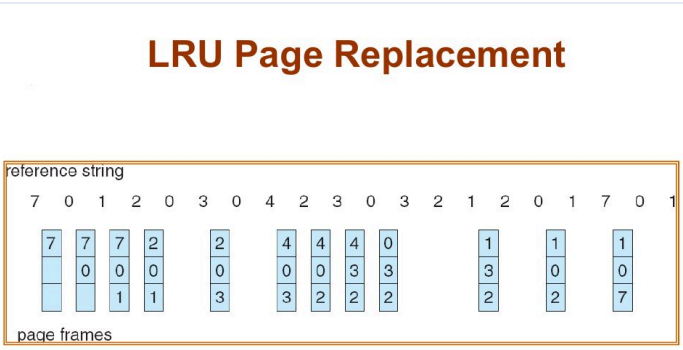
FIFO = รีดตัวใส่นานสุดก่อน · Optimal = หมอดู รีดตัวที่จะไม่ใส่อีกนานสุด · LRU = จำได้ว่าใส่ตัวไหนล่าสุดเมื่อไร รีดตัวที่ทิ้งไว้นานสุด



สไลด์ p.13 FIFO Page Replacement



สไลด์ p.16 Optimal Page Replacement



สไลด์ p.17 LRU Page Replacement

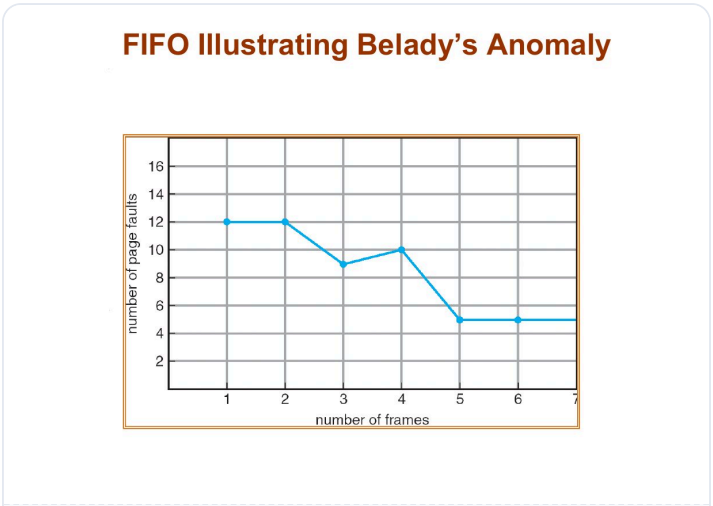
💡 เทคนิคไล่มือ: เขียน 3 ช่อง frame ใต้ string ทีละตัวอักษร · FIFO บอกตัวเลือก "เก่าสุด" ไว้ในหัว · LRU เขียนเลขกำกับ "ใช้ล่าสุดเมื่อไร" ใต้ช่อง · OPT กลับไปดูขวาของ string

I4. Belady's Anomaly — เพิ่ม frame แล้ว fault กลับโต! 🤯

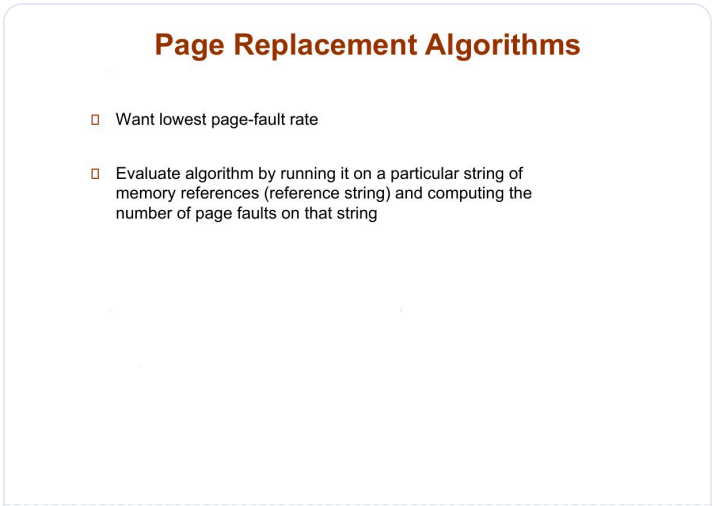
String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 (FIFO):

จำนวน frame	3 frames	4 frames
Page faults (เหตุการณ์เพจไม่อยู่ใน RAM)	**9**	**10** 😱

เพิ่ม RAM fault กลับเพิ่ม — เกิดได้กับ **FIFO** (และ non-stack algorithm) แต่**ไม่เกิดกับ LRU / Optimal**



สไลด์ p.14 FIFO Illustrating Belady's Anomaly



สไลด์ p.11 Page Replacement Algorithms (กราฟ fault vs frames)

I5. EAT เมื่อมี Page Fault Rate (p)

$$EAT = (1 - p) \times \text{memory access time} + p \times \text{page fault service time}$$

ตัวเลขชวนคิด: mat = 200ns, fault service = 8ms → ที่ p = 0.001 (fault แค่ 0.1%) EAT = 0.999×200 + 0.001×8,000,000 ≈ 8,200ns = 8.2µs — ช้าขึ้น 40 เท่า! ห้ามให้ p โต

I6. Global vs Local Allocation

- **Global replacement**: แทนที่ได้จาก **frame ใดก็ได้ในระบบ** — แย่งข้าม process ได้
- **Local replacement**: เลือกจาก**เฉพาะ frame ของตัวเอง** (คนละบ้านกับคนละบ้าน)

I7. Thrashing · Locality · Working Set (ชุดเพจที่ต้องมีใน RAM) 🔥

- **Thrashing**: process มีเพจไม่พอ → สลับเพจเข้า-ออก disk ตลอดเวลา → **CPU utilization ดิ่ง** · OS เข้าใจผิดว่า "งานน้อย" เลยเพิ่ม process → thrashing (สลับเพจหนักจนระบบแย่) ยิ่งหนัก (วงจรนรก) · เกิดทั้งข้ามงานและในงานเดียว (ลูปข้ามเพจ)
- **Locality of reference**: โปรแกรมมักอ้างกลุ่มเพจเดิมซ้ำ ๆ ในช่วงสั้น ๆ (อยู่ในฟังก์ชัน/ลูปเดิม)
- **Working set**: กลุ่มเพจที่ process ต้องมีใน RAM เพื่อไม่ให้ fault ในช่วง locality นั้น — มองย้อนหลัง Δ (working-set window เช่น 10,000 instruction) · **WSS_i** = จำนวนเพจที่ P_i อ้างใน Δ · Δ เล็กไปครอบ locality ไม่หมด / ใหญ่ไปกินหลาย locality / Δ = ∞ คือทั้งโปรแกรม
- **$D = \Sigma WSS_i$** = ความต้องการ frame รวม · ถ้า **D > m (frame จริง)** → **thrashing!** → OS ต้อง **suspend บาง process** คืน frame ให้คนที่เหลือ

> 🍳 **อุปมา**
> Working set = ของที่ต้องมีบนเคาน์เตอร์เพื่อทำเมนูให้จบ ถ้าวางไม่ครบ คุณวิ่งไปตู้เย็น (disk) ทุก 30 วินาที = ไม่มีอะไรเสร็จ → ทางแก้: ลดจำนวนเมนูที่ทำพร้อมกัน (suspend process)

Thrashing

- **Thrashing** ≡ a process is busy swapping pages in and out
- If a process does not have “enough” pages, the page-fault rate is very high. This leads to:
  - low CPU utilization
  - operating system thinks that it needs to increase the degree of multiprogramming
  - another process added to the system

สไลด์ p.19 Thrashing

Working-Set Model

- $\Delta \equiv$ working-set window ≡ a fixed number of page references
  Example: 10,000 instruction
- WSS_i (working set of Process P_i) =
  total number of pages referenced in the most recent Δ (varies in time)
  - if Δ too small will not encompass entire locality
  - if Δ too large will encompass several localities
  - if $\Delta = \infty \Rightarrow$ will encompass entire program
- $D = \Sigma\ WSS_i \equiv$ total demand frames
- if $D > m \Rightarrow$ Thrashing
- Policy if D > m, then suspend one of the processes

สไลด์ p.22 Working-Set Model (D = ΣWSSᵢ, D > m → suspend)

I8. โจทย์สุดคลาสสิก: Array 128×128 🔥 (สไลด์ p.24)

`int data[128][128]` · **1 แถว = 1 เพจ** · เก็บแบบ **row-major** (เรียงตามแถว)

	ลูป	พฤติกรรม	Page faults
❌ **แย่**	`for (j...) for (i...) data[i][j]=0;` (column นอก, row ใน)	ทุกครั้งกระโดดข้ามแถว = เปลี่ยนเพจทุกครั้ง	**128 × 128 = 16,384**
✅ **ดี**	`for (i...) for (j...) data[i][j]=0;` (row นอก, column ใน)	อยู่เพจเดียวกันทั้งแถว fault เฉพาะตอนเปลี่ยนแถว	**128**

โค้ดเหมือนกัน สลับลูปเท่านั้น → `fault` ต่างกัน 128 เท่า (16,384 : 128)

บทเรียน: เขียนลูปให้เข้าถึงข้อมูลต่อเนื่องตาม row-major = locality ดี

J. File System (OSNo10)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: File System = ผู้จัดการไฟล์บนดิสก์ — ตั้งชื่อ เก็บ ดึง แก้ไข จัด metadata และกำหนดสิทธิ์การเข้าถึง โดยที่เราไม่ต้องรู้เลยว่าข้อมูลอยู่เซกเตอร์ไหนของฮาร์ดดิสก์

ความสำคัญกับข้อสอบ: เทียบ FAT32 (ระบบไฟล์เก่า ไฟล์เกิน 4GB ไม่ได้)/NTFS (ระบบไฟล์ของ Windows)/ext4/exFAT (FAT32 ตัวใหม่ ไฟล์ใหญ่ได้)/APFS (ระบบไฟล์ของ Apple), วิธีจัดสรรบล็อก 3 แบบ, คำนวณ inode (โครงสร้างเก็บข้อมูลไฟล์ของยูนิกซ์), hard link vs symbolic link, สิทธิ์ rwx → ส่วนใหญ่ออกถูก/ผิด + โจทย์คำนวณสั้น ๆ

ใช้จริงในคอมพิวเตอร์: เลือกฟอร์แมตแฟลช (FAT32 เก็บไฟล์เกิน 4GB ไม่ได้!), คำสั่ง chmod/ls -l ที่เจอทุกวันใน Linux, สร้าง shortcut ไฟล์ = symbolic link — ใช้ในชีวิตประจำวันจริง

J1. File System รับผิดชอบอะไร

จัดการเก็บ/ดึง/จัดระเบียบข้อมูลบน storage — หน้าที่: **ตั้งชื่อ+โครงสร้างไฟล์ · เก็บข้อมูล · ดึง/แก้ไข · Metadata · ความปลอดภัย/สิทธิ์เข้าถึง** · หัวใจคือ **abstraction**: interface เข้าถึงไฟล์เหมือนกันไม่ว่าข้อมูลอยู่บน HDD/SSD/USB (device independence — จาก Quizizz 4 ข้อ 2)

J2. 5 ระบบไฟล์ยอดนิยม

ระบบไฟล์	ไฟล์สูงสุด	Volume สูงสุด	Journaling (สมุดบันทึกเพื่อกู้คืนหลังพัง)	ใช้ที่ไหน
FAT32 (1996)	**4 GB** 😅	8 TB	ไม่มี	USB/การ์ด — เข้ากันได้*ทุก*เครื่อง
NTFS (1993)	16 TB	256 TB	✅	Windows — สิทธิ์/เข้ารหัส (EFS)/compression/quota
ext4 (2008)	16 TB	1 EB	✅	Linux หลัก (ต่อจาก ext3)
exFAT (2006)	16 EB	128 PB	ไม่มี	USB/SD ใหญ่ข้าม OS — แก้ขีดจำกัด 4GB
APFS (2017)	8 EB	8 EB	✅ + Snapshots	Apple ทั้งหมด — เข้ารหัสทั้งดิสก์

🎯 จาก Quizizz 4

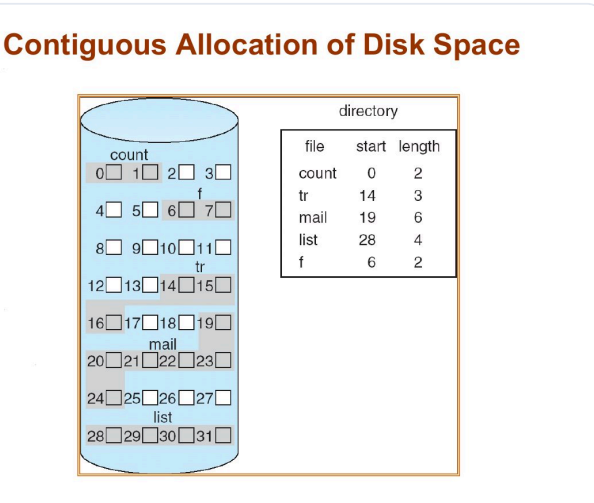
• USB/external drive = **ExFAT** · • **Journaling ≠ เข้ารหัส!** — journaling คือบันทึก log ก่อนเขียนจริงเพื่อ**กู้คืนหลัง crash** · เข้ารหัสของ NTFS คือ **EFS** แยกกัน · • FAT32 ไฟล์เดี่ยว**ต้น 4GB เด็ดขาด** (ฟิลด์ 32 บิต) ดิสก์ว่างเยอะก็ไม่ช่วย

J3. File (ไฟล์) Allocation Methods 🔥 (วิธีจัดสรรบล็อกให้ไฟล์)

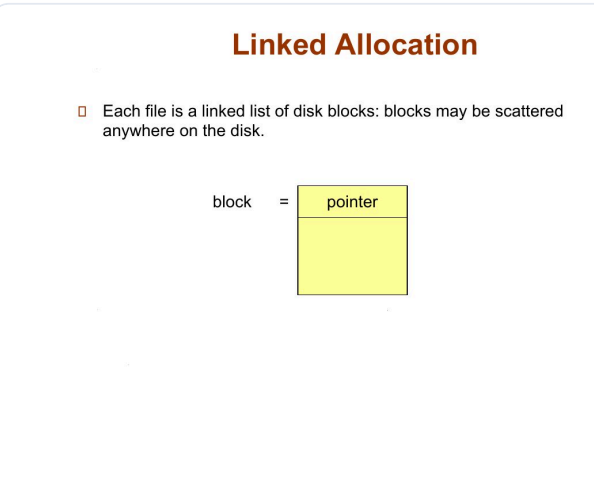
วิธี	หลักการ	ข้อดี	ข้อเสีย
Contiguous	บล็อก**ติดกันเป็นผืน** · directory เก็บ start + length	อ่านเร็วมาก (ไม่ต้อง seek)	**External fragmentation** + ขยายไฟล์ยาก (บล็อกถัดไปถูกจอง = ต้องย้ายทั้งไฟล์)
Linked	บล็อกกระจาย + pointer (ตัวชี้ตำแหน่ง) ชี้บล็อกถัดไปเป็นโซ่ · directory เก็บ start + end	ไม่มี external frag · ขยายง่าย	**Random access ช้ามาก** (ไล่โซ่) · pointer เปลืองพื้นที่ · pointer พัง = ท่อนหลังหาย
Indexed	รวม pointer ทุกตัวไว้ใน **Index Block** · directory ชี้มาจุดเดียว	**Random access เร็ว** · ไม่มี external frag	เปลืองบล็อก index (ไฟล์จิ๋วก็เสีย 1 บล็อก) · ไฟล์ใหญ่ต้อง multi-level

🎯 FAT (File Allocation Table (ตาราง))

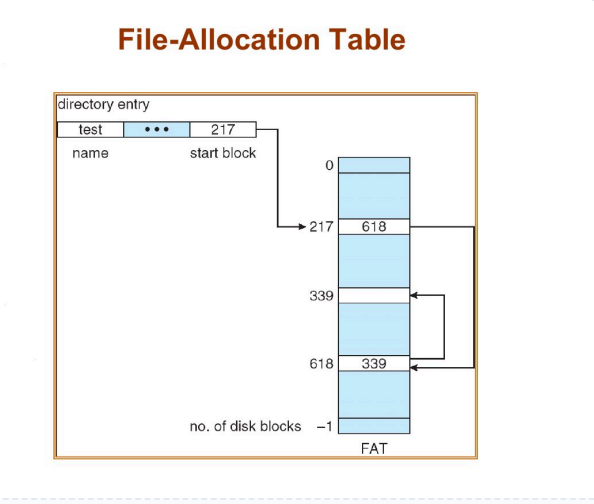
FAT = ดึง pointer ทั้งหมดออกจาก data block มารวมไว้ใน**ตารางเดียว (อยู่ใน RAM)** → ไล่โซ่ได้เร็วโดยไม่ต้องเปิดดิสก์ทีละบล็อก — นี่คือที่มาของชื่อ FAT filesystem



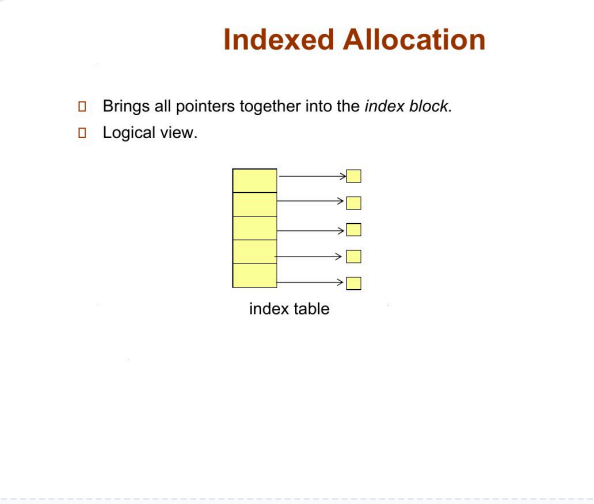
สไลด์ p.19 Contiguous Allocation (การจัดสรรบล็อกติดกัน)



สไลด์ p.20 Linked Allocation (การจัดสรรแบบโซ่ pointer)



สไลด์ p.24 File-Allocation Table



สไลด์ p.25 Indexed Allocation (การจัดสรรแบบสารบัญ)

J3.1 คำนวณ index / inode 🔥 (สูตรเดียวใช้ทุกโจทย์)

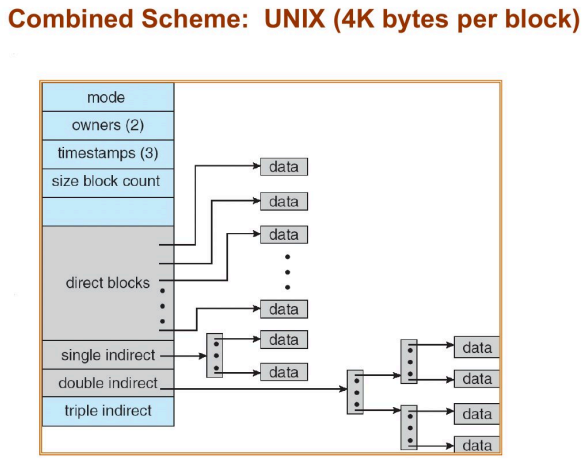
`Entries ต่อ 1 index block = Block Size ÷ Pointer Size`

จาก Quizizz 4: บล็อก 2KB ÷ pointer 4B = **512 entries** → ไฟล์ใหญ่สุด single-level = 512 × 2KB = 1MB

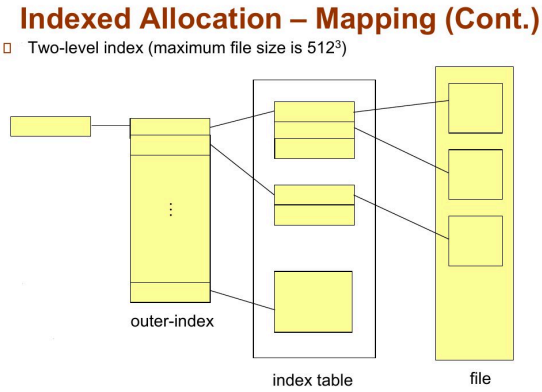
UNIX Combined Scheme — inode (สไลด์ p.29, block 4KB, pointer 4B): มี pointer 15 ตัว

Pointer	จำนวน	คำนวณ	ขนาด
Direct ชี้ data ตรง ๆ	12	12 × 4KB	**48 KB**
Single indirect	1	(4KB÷4B) × 4KB = 1024 × 4KB	**4 MB**
Double indirect	1	1024^2 × 4KB	**4 GB**
Triple indirect	1	1024^3 × 4KB	**4 TB**
ไฟล์ใหญ่สุด ≈ 48KB + 4MB + 4GB + 4TB			**≈ 4 TB**

> **🎯 จุดพลาด**
> ลืมหา "1024 ช่องต่อบล็อก" ก่อน / ลืมบวก direct 48KB / บวกเลขผิดหลัก (4MB ≠ 4×1024KB=4096KB ก็คือ 4MB ตรงกัน แต่ double = 1024×1024×4KB = 4GB)



สไลด์ p.29 Combined Scheme: UNIX (4K/block) — 15 pointers (ตัวชี้ตำแหน่ง)



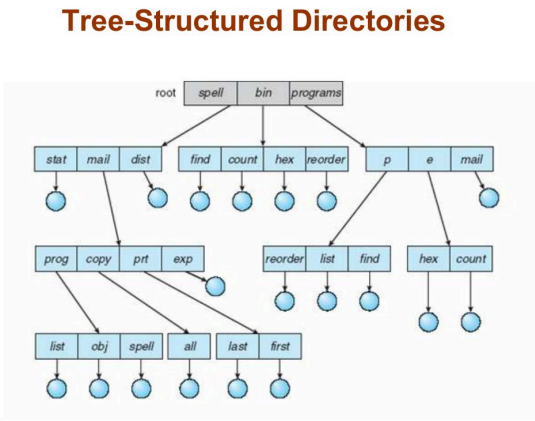
สไลด์ p.28 Two-level index (512^3 ในสไลด์ = block 512B)

J4. Directory (โฟลเดอร์/สารบัญไฟล์) · Tree vs Acyclic Graph

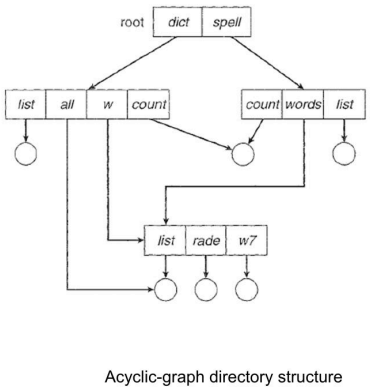
- **Directory** = symbol table แปลง "ชื่อไฟล์ → ข้อมูลไฟล์" · operations: search, create, delete, list, rename, traverse
- **Tree**: ไฟล์มี parent เดียว — ปลอดภัยแต่**แชร์ไม่ได้**
- **Acyclic graph** (ไม่มีวงวน): แชร์ได้ผ่าน **link** — ปัญหา: aliasing (ไฟล์เดียวหลายชื่อ), การลบไฟล์แชร์

การลบไฟล์แชร์ 2 แนวทาง

แนวทาง	ทำอย่างไร	ข้อควรระวัง
ลบเมื่อไรก็ได้	ลบไฟล์ทันทีที่ใครสั่ง	**Dangling pointer** — link ที่เหลือชี้หาอะไรที่ไม่มีแล้ว (แนวทาง symbolic link)
เก็บจน reference หมด	นับ **reference count**: เพิ่ม link → count+1, ลบ link → count−1, **count = 0 ค่อยปล่อยพื้นที่**	ต้องตรวจว่ากราฟไม่มีวงวน (cycle) — แนวทาง hard link ของ UNIX



สไลด์ p.32 Tree-Structured Directories



สไลด์ p.33 Acyclic-graph Directory

J5. Hard Link (ลิงก์ชี้ inode เดียวกัน) vs Symbolic (Soft) Link (ลิงก์ (ทางเชื่อม)) 🔥

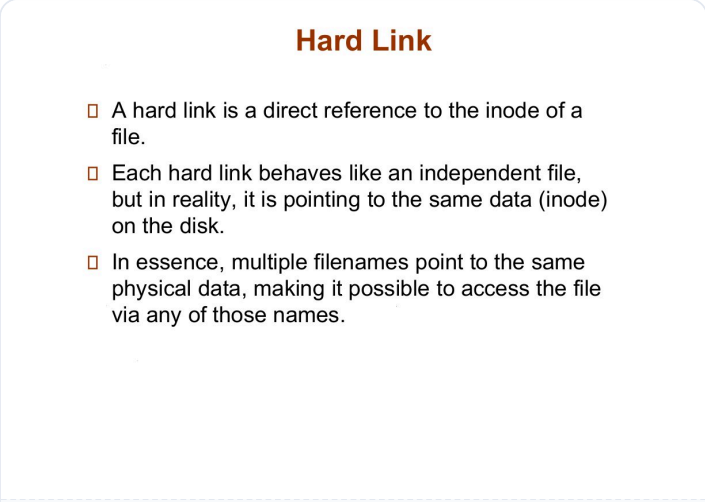
	Hard Link `ln original new`	**Symbolic Link (ลิงก์แบบเก็บ path (shortcut))** `ln -s original new`
ชี้ไปที่	directory entry ชี้**inode เดียวกัน**โดยตรง + เพิ่ม link count	**ไฟล์ใหม่แยกต่างหาก** เก็บ "path" ของต้นทาง (shortcut)
ลบต้นฉบับแล้ว	ข้อมูลยังอยู่ — ปล่อยจริงเมื่อ **link count = 0**	กลายเป็น **broken/dangling link**
ข้าม partition/FS	❌ ห้าม	✅ ได้
ชี้ directory	❌ ห้าม (กัน cycle)	✅ ได้

🏠 อุปมา

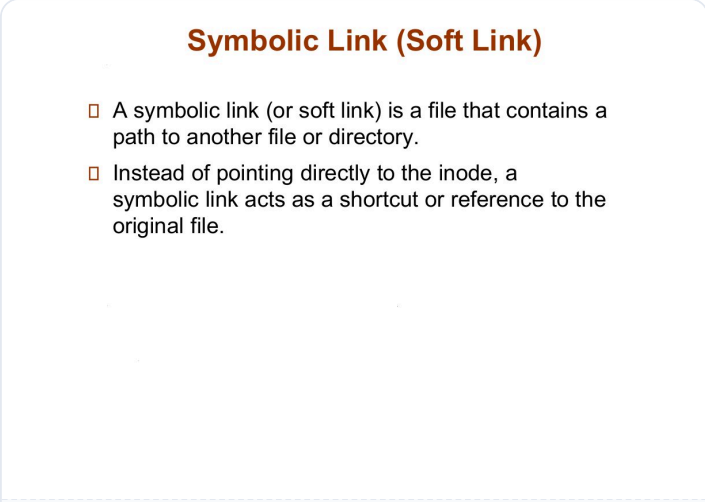
Hard link = ลงทะเบียนบ้านเพิ่มอีกชื่อ — บ้านหลังเดิม อยู่ต่อจนคนทะเบียนหมด · Soft link = กระดาษโน้ตเขียนว่า "บ้านอยู่ซอย 5" — โยนบ้านทิ้ง กระดาษก็ชี้อากาศ

🎯 จาก Quizizz 4

• Symbolic link อีกชื่อคือ **soft link** ไม่ใช่ hard link · • Hard link จัดการด้วย **Reference Counter** ✓ · • **ไม่มีคำสั่ง hardln!** — จริงคือ `ln` (hard) / `ln -s` (symbolic)



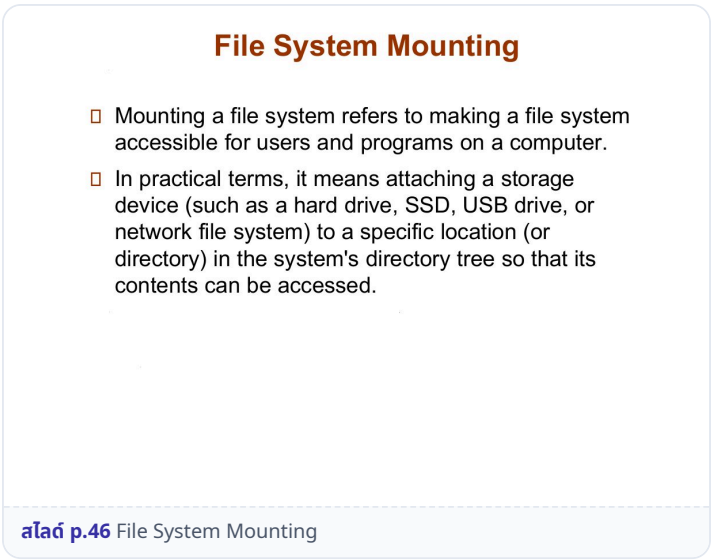
สไลด์ p.42 Hard Link



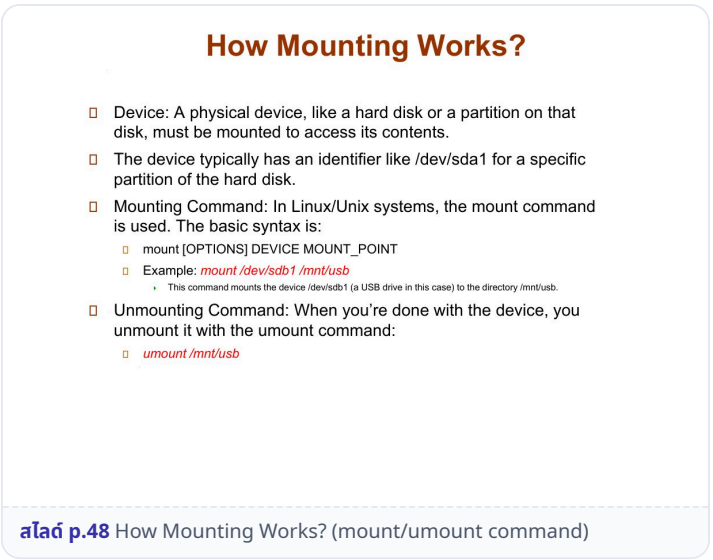
สไลด์ p.44 Symbolic Link (Soft Link)

J6. Mounting (การติดตั้งดิสก์เข้าระบบไฟล์)

- **Mounting** = ติด filesystem ของอุปกรณ์ (partition/USB/network) เข้า directory tree ที่ **mount (ติดตั้งเข้าระบบไฟล์) point** เช่น `mount /dev/sdb1 /mnt/usb`
- ก่อนถอดต้อง `umount` — ให้การเขียนค้าง ๆ เสร็จก่อน ไม่งั้นข้อมูลพัง
- จาก Quizizz 4: อุปกรณ์เดียว **mount ได้หลาย mount point** (ข้อความว่า "ไม่สามารถ" = เท็จ)



สไลด์ p.46 File System Mounting



สไลด์ p.48 How Mounting Works? (mount/umount command)

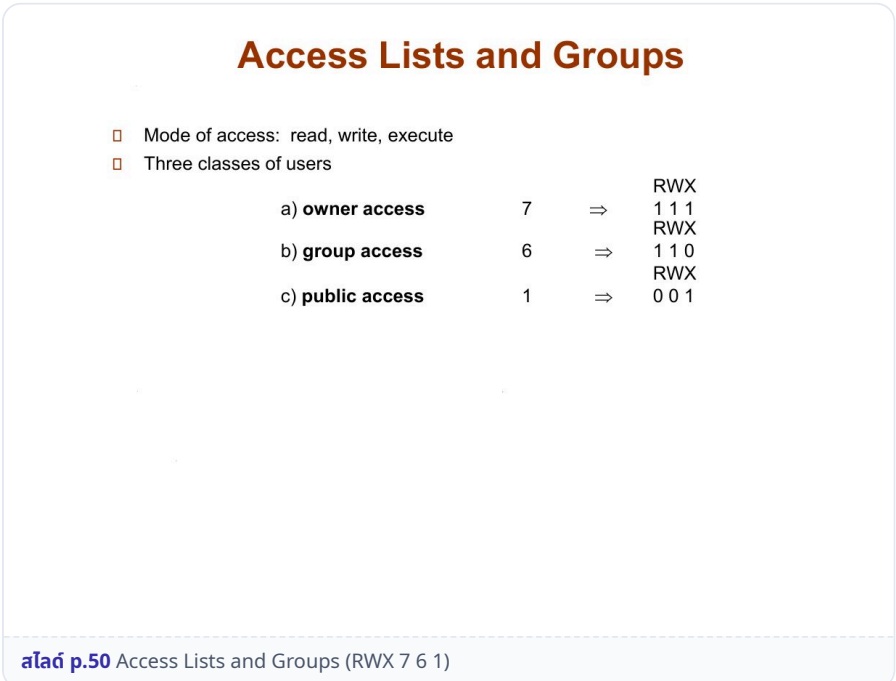
J7. Protection — สิทธิ์ RWX 🔥

เจ้าของไฟล์คุมได้ 2 อย่าง: **what can be done** (read/write/execute) กับ **by whom** (owner/group/others)

$$R = 2^2 = 4 \cdot W = 2^1 = 2 \cdot X = 2^0 = 1$$

3 กลุ่ม: owner, group, other (public) — ตัวอย่างสไลด์ p.50: owner=7 (rwx), group=6 (rw-), public=1 (--x) = 761

เลข	บวกได้จาก	แปลง
7	4+2+1	rwx — ทุกสิทธิ์
6	4+2	rw-
5	4+1	r-x (ไม่ใช่ 6! จาก Quizizz)
4	4	r--
0	—	--- ไม่มีสิทธิ์



สไลด์ p.50 Access Lists and Groups (RWX 7 6 1)

K. ความรู้จาก Quizizz ในห้องเรียน (4 ชุด) — จัดเรียงตามชุด

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: ความรู้ที่สกัดจากแบบควิซในห้องเรียนจริง 4 ชุด (Signal/Pipe, Shared Memory (หน่วยความจำที่แชร์กัน), Semaphore, File Management) — ทุกข้อมี "ความรู้สำคัญ" ที่ตอบโจทย์แบบเดิมได้ทันที

ความสำคัญกับข้อสอบ: นี่คือธนาคารข้อสอบย่อที่ตรงสไตล์อาจารย์ที่สุด — หลายข้อมีกับดักที่อาจารย์ตั้งใจหลอก (เช่น ข้อ 8 ชุด 4 มีคีย์ขัดตำรา — ต้องตอบ False ตามคีย์!) อ่านก่อนสอบคุ้มที่สุด

ใช้จริงในคอมพิวเตอร์: ประเด็นในควิซคือข้อผิดพลาดจริงที่โปรแกรมเมอร์พลาดบ่อย เช่น เข้าใจว่า signal() ใช้ส่งสัญญาณ, ลืมปิดปลาย pipe (ท่อส่งข้อมูลทางเดียว) — รู้ไว้เขียนโค้ดจริงไม่พัง

ทุกข้อเป็นความรู้ที่ตอบโจทย์แบบเดิมได้ทันที — อ่านกล่อง "ความรู้สำคัญ" ต่อข้อ

K1. Quiz 1 — IPC & Unix Signal

ข้อ 2 False

"Signal ใช้ส่งข้อความระหว่าง process"

ความรู้: Signal = **event notification** เท่านั้น — ไม่มี payload แนบข้อมูลไม่ได้ ผู้รับรู้แค่ว่า "signal ชนิด X มา" · ส่งข้อมูลจริงใช้ Message Passing (การส่งข้อความผ่าน OS) / Shared Memory

ข้อ 3 Message Passing

"IPC ที่ OS จัดจังหวะการรับส่งให้"

ความรู้: Message passing ให้ OS จัดทั้ง send/receive และ**จังหวะ** (blocking/non-blocking) · Shared Memory จัดเตรียมแค่พื้นที่ — **ซิงก์จังหวะเป็นหน้าที่ของ process เอง** (เช่นใช้ semaphore)

ข้อ 4 SIGINT

"Ctrl-C ส่ง signal อะไร"

ความรู้: Ctrl-C → **SIGINT** (INT = INTerrupt) · คู่เทียบ: SIGFPE = หารศูนย์, SIGTERM = ขอจบสุภาพ (default ของ kill (ส่งสัญญาณไปยังกระบวนการอื่น)), SIGQUIT = Ctrl-\, SIGKILL = ฆ่าทันทีจับไม่ได้ (9)

ข้อ 5 False

"signal() ใช้ส่ง signal ระหว่าง process"

ความรู้: `signal()` = **ลงทะเบียน handler (ฟังก์ชันรับมือเหตุการณ์)** ("โดน X แล้วทำอะไร") · ตัวส่งคือ `kill(pid, sig)` (ส่งข้าม process) และ `raise(sig)` (ส่งให้ตัวเอง)

ข้อ 6 False

"1 process จัดการ signal ได้แค่ 1 ตัว"

ความรู้: จัดการได้**หลาย signal** — แต่ละชนิดมี handler ของตัวเอง · ข้อจำกัดจริง: 1 ชนิด มี 1 handler (ลงทะเบียนซ้ำ = ทับตัวเดิม)

ข้อ 7 False

"kill() ใช้ส่งได้แค่ SIGKILL"

ความรู้: System (ระบบ) call `kill(pid, sig)` ส่งได้**ทุกชนิด** — SIGKILL แค่ 1 ตัวเลือก (ชื่อมาจากประวัติศาสตร์ ทำให้เข้าใจผิด)

ข้อ 8 True

"คำสั่ง kill ใน UNIX ใช้ส่ง signal ให้ process"

ความรู้: หน้าที่จริงของคำสั่ง kill = **ส่ง signal** (default SIGTERM) · `kill -9 pid` = SIGKILL · ⚠️ แยกจากข้อ 7: system call ส่งได้ทุกชนิด (โจทย์ใส่ "เท่านั้น" มาหลอก)

K2. Quiz 2 — Shared Memory & Message Passing (Pipe)

ข้อ 2 False

"Message passing รองรับทิศทางเดียว"

ความรู้: ทิศเดียวจริง ๆ คือ **pipe ธรรมดา (anonymous)** — สองทิศต้อง 2 pipe · Message passing แต่ละ process มีทั้ง send/receive = **สองทิศได้**

ข้อ 3 อาเรย์ int 2 ตัว

"พารามิเตอร์ของ pipe()"

ความรู้: `int pfd[2]; pipe(pfd);` — OS ใส่ค่า 2 ช่อง: **pfd[0] = ปลายอ่าน, pfd[1] = ปลายเขียน** (file (ไฟล์) descriptor ไม่ใช่ชื่อไฟล์ — pipe ธรรมดาไม่มีชื่อ)

ข้อ 4 write(pfd[1], data, length)

"ประโยคส่งข้อมูลเข้า pipe"

ความรู้: pipe ใช้ system call ไฟล์มาตรฐาน: **write()** ส่ง / **read()** รับ · เขียนต้องลงปลายเขียน **pfd[1]** · จำ: "write to 1, read from 0"

ข้อ 5 dup2(pfd[1], 1)

"เปลี่ยนทิศ stdout ไปปลายเขียน pipe"

ความรู้: `dup2(oldfd, newfd)` = ทำให้ **newfd** ชี้ที่เดียวกับ oldfd · fd 0=stdin, 1=stdout, 2=stderr → stdout(1) ไหลเข้าปลายเขียน = `dup2(pfd[1],1)` · pattern มาตรฐาน: ฝั่งเขียน dup2(pfd[1],1) แล้ว exec / ฝั่งอ่าน dup2(pfd[0],0) · ปิดปลายที่ไม่ใช้ก่อน

ข้อ 6 shmget()

"System call ขอสร้าง shared memory"

ความรู้: shm**get** = get — ขอสร้าง/อ้างถึง segment ได้ **shmid** · (shmat=attach, shmdt=detach, shmctl=control/ลบ)

ข้อ 7 ftok()

"ฟังก์ชันสร้าง key ของ shared memory"

ความรู้: `key_t ftok(path, id)` — path + id เดียวกัน → key เดียวกัน → อ้าง shm ก้อนเดียวกันได้ (นิยม path = ".")

ข้อ 8 0764

"rwxrw-r-- เป็นเลขฐานแปดอะไร"

ความรู้: r=4 w=2 x=1 → rwx=7, rw-=6, r--=4 → **0764** (ใช้กับ shmget flag เช่น IPC_CREAT | 0666 ด้วย)

ข้อ 9 IPC_PRIVATE

"shmget ที่การันตีไม่ซ้ำกับใครแน่นอน"

ความรู้: IPC_PRIVATE = key พิเศษ OS สร้าง segment ใหม่ไม่ซ้ำใครเสมอ — นิยมใช้ parent→child (shmid ส่งต่อได้ในตระกูล) · IPC_EXCL แค่ flag กันชน ไม่การันตี

ข้อ 10 shared memory id

"shmget() คืนอะไร"

ความรู้: คืน **shmid** (เลขจำนวนเต็ม) — เอาไปใช้กับ shmat/shmctl/shmdt · flow: key → shmget → **shmid** → shmat → pointer

ข้อ 11 shmat()

"ฟังก์ชันให้ตัวแปรอ้างถึงตำแหน่ง shared memory"

ความรู้: shm**at** = attach — ผูก segment เข้า address space แล้ว **คืน pointer**: `shm_ptr = shmat(shmid, NULL, 0)`

ข้อ 12 False

"ตัวแปรเก็บตำแหน่ง shm ต้องเป็น pointer ไป int เท่านั้น"

ความรู้: shmat คืน **void*** — cast เป็นชนิดไหนก็ได้ (char*, struct data* ...) ข้อจำกัดเดียวคือต้องเป็น pointer

ข้อ 13 struct data *shm_ptr;

"shmget(..., sizeof(struct data), ...) ตัวแปรชี้ต้องประกาศยังไง"

ความรู้: จองขนาด sizeof(struct data) → pointer ต้องตรงชนิด: `struct data *p; p = (struct data *) shmat(shmid, NULL, 0);` ใช้ผ่าน `p->field`

🟢 Shared Memory ครบวงจร (จำ 6 ขั้น)

```
key_t key = ftok(".", 'a');                    // 1. สร้าง key
int shmid = shmget(key, sizeof(struct data), IPC_CREAT | 0666); // 2. สร้าง segment
struct data *p = (struct data *) shmat(shmid, NULL, 0);   // 3. attach
/* 4. อ่าน/เขียนผ่าน p */
shmdt(p);                                      // 5. detach
shmctl(shmid, IPC_RMID, NULL);                 // 6. ลบ (คนสุดท้าย)
```

K3. Quiz 3 — Semaphore

ข้อ 2 False

"Semaphore ที่ใช้กับ critical section คือ counting"

ความรู้: CS ใช้ **binary semaphore (mutex)** เริ่ม 1: wait→CS→signal · counting (เริ่ม = จำนวน resource) ใช้กับ resource หลายหน่วย เช่น bounded buffer

ข้อ 3 False

"Semaphore การันตีผ่านเงื่อนไข 3 ข้อของ CS"

ความรู้: 3 เงื่อนไข = Mutual Exclusion / Progress / Bounded Waiting — semaphore การันตีได้แค่ **ข้อ 1** · progress/bounded waiting ขึ้นกับ implementation (classic อาจ starvation)

ข้อ 4 2 ตัว

"bounded buffer ใช้ counting semaphore กี่ตัว"

ความรู้: ใช้รวม 3 ตัว: empty(n), full(0), mutex(1) — แต่**counting จริงมี 2** (empty, full) เพราะ mutex เป็น binary

ข้อ 5 empty

"producer ตรวจช่องว่างด้วยตัวไหน"

ความรู้: Producer: `wait(empty)` ขอช่องว่างก่อน · Consumer: `wait(full)` ก่อนอ่าน · mutex ใช้ตอนแตะ buffer จริง (ลำดับ: resource ก่อน mutex!)

ข้อ 6 mutex

"readers-writers คุมจำนวน reader ด้วย semaphore ไหน"

ความรู้: mutex ครอบช่วง readcount++/-- · **readcount เป็น int ธรรมดา** ไม่ใช่ semaphore · wrt คุม writer / reader แรกล็อก-ตัวสุดท้ายปลด

ข้อ 7 resource

"ตะเกียบใน Dining philosopher แทนอะไร"

ความรู้: ตะเกียบ = **shared resource** (นักปราชญ์ = process) · implement ใช้ semaphore 1 ตัวต่อ 1 ตะเกียบ แต่ตัวตนของมันคือ resource

ข้อ 8 False

"semaphore การันตีนักปราชญ์ทุกคนได้กิน"

ความรู้: หยิบซ้ายพร้อมกัน → รอขวาเป็นวง = **deadlock** + ไม่มี bounded waiting → starvation · แก้: จำกัด ≤ 4 คน / คนนึงหยิบขวาก่อน / ใช้ Monitor

ข้อ 9 sem_post()

"PThread semaphore ตัวไหน = signal()"

ความรู้: PThread: `sem_init(&s,0,val)` สร้าง · **sem_wait = P (ลด)** · **sem_post = V (เพิ่ม/ปลุก)** · sem_destroy เก็บกวาด · **ไม่มี sem_signal ใน POSIX!**

K4. Quiz 4 — Introduction to File Management

ข้อ 2 True

"เข้าถึงไฟล์เหมือนกันไม่ว่าอุปกรณ์อะไร"

ความรู้: ระบบไฟล์ = ชั้น abstraction → open/read/write ชุดเดียวกันทุกอุปกรณ์ (**device independence**)

ข้อ 3 False

"Journaling = การเข้ารหัสไฟล์"

ความรู้: Journaling = write-ahead log เพื่อ**กู้คืนหลัง crash** · การเข้ารหัสเป็นฟีเจอร์แยก (NTFS ใช้ **EFS**)

ข้อ 4 ExFAT

"ระบบไฟล์สำหรับ USB drive"

ความรู้: USB/external = **ExFAT** (ใหญ่กว่า 4GB ข้าม Windows-macOS) · NTFS=Windows, ext4=Linux, APFS=Apple

ข้อ 5 False

"contiguous ปัญหาคือ internal fragmentation"

ความรู้: contiguous → **external** fragmentation (ต้องกันบล็อกติดกัน พื้นที่แตกเป็นหลุม) · internal = เศษในบล็อกสุดท้ายที่ไม่เต็ม

ข้อ 6 False

"linked allocation เป็นการจัดสรรแบบต่อเนื่อง"

ความรู้: linked = บล็อกกระจาย + pointer โซ่ = **non-contiguous** โดยนิยาม (FAT อยู่ตระกูลนี้)

ข้อ 7 False

"บล็อกใหญ่ → external fragmentation"

ความรู้: บล็อกใหญ่ → เศษในบล็อกสุดท้ายเยอะ = **internal fragmentation** (ไฟล์ 1B ในบล็อก 4KB = เสีย ~4KB)

ข้อ 8 False (ตามคีย์!)

"single-level index: ขนาดไฟล์สูงสุดถูกจำกัดด้วยจำนวนช่องใน index block"

ความรู้: ⚠️**คีย์ผู้สอน = False** (แม้ตามตำราน่าจะ True) — ตอบ False ตามคีย์ · ความรู้จริง: ไฟล์ใหญ่สุด = entries × block size

ข้อ 9 512

"บล็อก 2KB, pointer 4B → 1 index block มีกี่ช่อง"

ความรู้: entries = 2048 ÷ 4 = **512** → single-level ใหญ่สุด 512 × 2KB = 1MB · สูตรเดียวกับ inode ของ J3.1

ข้อ 10 False

"Symbolic link เรียกอีกอย่างว่า hard link"

ความรู้: symbolic = **soft link** (ไฟล์แยกเก็บ path) · hard link = entry ชี้ inode เดียวกัน

ข้อ 11 True

"Hard link จัดการด้วย Reference Counter"

ความรู้: inode เก็บ **link count** — rm แค่ลด counter ปล่อยพื้นที่เมื่อ = 0

ข้อ 12 False

"คำสั่ง hardln ใช้สร้าง Hard Link"

ความรู้: ไม่มี hardln! จริงคือ `ln target link` (hard) / `ln -s target link` (symbolic)

ข้อ 13 False

"mount อุปกรณ์เดียวกันได้แค่ 1 mount point"

ความรู้: mount อุปกรณ์เดียวได้**หลาย mount point** (รองรับ bind mount) — "ไม่สามารถ" เป็นเท็จ

L. แผ่นสูตรเร็วท้ายเล่ม (อ่าน 3 นาทีก่อนเข้าห้อง)

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: สรุปสูตรคำนวณทุกบทมารวมใน 1 หน้า (TAT/WT, Paging, EAT, นับ fault, Banker's, inode, rwx)

ความสำคัญกับข้อสอบ: เป็น index ระหว่างทำข้อสอบ — ตันหัวข้อไหนกลับมาเปิดหน้านี้ก่อน แล้วค่อยไล่ไปส่วนเต็มที่อยู่หน้าก่อน ๆ

ใช้จริงในคอมพิวเตอร์: สูตรพวกนี้คือสิ่งที่ OS คำนวณจริงทุกมิลลิวินาที (แปลง address ทุกครั้งที่เข้าถึง RAM) — เข้าใจสูตร = เข้าใจการทำงานเครื่อง

⏱️ CPU Scheduling

TAT	Completion − Arrival
WT	TAT − Burst (หา TAT ก่อนเสมอ!)
RR	รอไม่เกิน (n−1)q · q ใหญ่=FCFS
เท่ากัน	เลือกมาถึงก่อน (FCFS tiebreak)

🧮 Paging

offset	$\log_2$(page size) บิต
page#	บิต address − บิต offset
เพจ	ceil(ขนาด ÷ page size) — เริ่มนับ 0
PA	(frame × pageSize) + offset
EAT	2 + ε − α (ตย. ε=0.1, α=0.8 → 1.3μs)
2-level	inner = ceil(เพจ ÷ 1024) × 4KB + 4KB

💾 Page Replacement (7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1 / 3 frames)

FIFO	15	เข้าก่อนออกก่อน · มี Belady (3f=9, 4f=10)
OPT	9	มองอนาคต ใช้จริงไม่ได้
LRU	12	มองอดีต ใช้จริงได้
EAT(p)	(1−p)×mat + p×service	
128×128	row นอก = 128 · col นอก = 16,384	

🏦 Banker's · 📁 File

Need	Max − Allocation
ให้ได้ไหม	Req ≤ Need และ ≤ Avail แล้ว safety ใหม่
seg	d < Limit → PA = Base + d (≥ = trap)
fit	First=หลุมแรก Best=เหลือน้อยสุด Worst=ใหญ่สุด
inode	entries = block÷ptr (4K/4B=1024) · 48KB+4MB+4GB+4TB
rwx	r=4 w=2 x=1 · 761 = rwx rw- --x

🚨 กลยุทธ์ห้องสอบ (จากรีวิวรุ่นพี่)

• ถูก/ผิด: ถูก +2 ผิด −1 ไม่ตอบ 0 — **ไม่แน่จริงอย่าตอบ** · • ทำโจทย์คำนวณ (Paging/Replacement (การแทนที่)/Scheduling) **ก่อนข้อทฤษฎี** คะแนน/นาทีสูงกว่า · • จับเวลาข้อคำนวณ ≤ 2 นาที ติดให้ข้าม · • เขียนขั้นตอนสั้น ๆ ข้างคำตอบ · • โค้ดทุกข้อ compile ผ่านหมด — ไม่มีตอบ Error!